

TEXTBOOK EDITION • PYTHON & DATA ENGINEERING

Python Data Ecosystem & Analytics

NumPy Vectorization, pandas DataFrames, Matplotlib & Banking Analytics Pipeline

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	12 Hours
DIFFICULTY LEVEL	Advanced Core
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)	Study Time: 35m
Chapter 02 — Python Fundamentals, OOP, Internals & Advanced Concepts	Study Time: 35m
Chapter 03 — NumPy: Numerical Computing & Vectorized Operations	Study Time: 35m
Chapter 04 — Complete pandas Mastery: Data School Q&A Curriculum (38 Videos)	Study Time: 35m
Chapter 05 — Matplotlib: Data Visualization & Financial Plotting	Study Time: 35m
Chapter 06 — SQL pandas Rosetta Stone: Translating SQL Knowledge to pandas	Study Time: 35m
Chapter 07 — The Complete Python Data Analysis Workflow: NumPy pandas Matplotlib	Study Time: 35m
Chapter 08 — Most Asked & Tricky Python Interview Questions	Study Time: 35m
Chapter 09 — Last-Minute Python Data Ecosystem Cheatsheet	Study Time: 35m
Chapter 10 — pandas Last-Minute Revision: High-Yield Cram Sheets	Study Time: 35m

CHAPTER 01

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

1. Why This Matters

In banking and financial backend systems, an algorithm that is $O(N^2)$ instead of $O(N \log N)$ can freeze a settlement engine during peak trading or UPI clearing windows. At IDFC FIRST Bank's Strategic Projects, engineers write code handling tens of thousands of concurrent payment requests. In interviews, stating an algorithm's complexity is not enough: interviewers will ask you to mathematically derive worst-case, average-case, and amortized bounds, identify auxiliary memory allocations, and analyze garbage collection overhead.

2. Prerequisites

- Basic algebra (logarithms, exponents, summation formulas: $\sum_{i=1}^n i = n(n+1)/2$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).

3. Concept

Asymptotic Notations

- **Big-O (O):** Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $0 \leq f(N) \leq c * g(N)$ for all $N \geq N_0$. Represents the worst-case scenario.
- **Big-Omega (Ω):** Lower bound. $f(N) \geq c * g(N)$ for all $N \geq N_0$. Represents best-case performance.
- **Big-Theta (Θ):** Tight bound. When an algorithm's upper bound and lower bound match: $c_1 * g(N) \leq f(N) \leq c_2 * g(N)$.
- **Little-o (o):** Strict upper bound where $f(N)$ grows strictly slower than $g(N)$.

Growth of Common Time Complexities:

$O(1)$ [Constant: Hash table lookup, Array indexing]

↓

$O(\log N)$ [Logarithmic: Binary search, Balanced BST lookup]

↓

$O(N)$ [Linear: Single scan, Two Pointers, Sliding Window]

↓

$O(N \log N)$ [Linearithmic: MergeSort, QuickSort (avg), Timsort]

↓

$O(N^2)$ [Quadratic: Nested loops, BubbleSort, Brute force pairs]

↓

$O(2^N)$ [Exponential: Naive Fibonacci, Subsets recursion]

↓

$O(N!)$ [Factorial: Permutations generation, Traveling Salesperson]

Amortized Analysis

When an operation is occasionally very expensive but predominantly cheap, the *amortized cost* averages the total cost over a long sequence of operations.

The Accounting Method: You overcharge cheap operations (e.g., normal array append = 1 unit of work charged as 3 units) and store the surplus as credit. When the expensive resizing operation occurs (copying N elements = N units of work), the accumulated credit pays for the reallocation. Thus, the amortized cost per append is strictly $O(1)$.

4. Simple Example: Linear vs Logarithmic Search

```
# Linear Search: O(N) Time, O(1) Space
def linear_search(arr: list[int], target: int) → int:
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Binary Search: O(log N) Time, O(1) Space
def binary_search(sorted_arr: list[int], target: int) → int:
    left, right = 0, len(sorted_arr) - 1
    while left ≤ right:
        mid = left + (right - left) // 2 # Prevents integer overflow
        if sorted_arr[mid] == target:
            return mid
        elif sorted_arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

5. Real-World Banking Example: High-Throughput Transaction Deduplication

Suppose an incoming UPI payment batch has $N = 100,000$ transactions. You must verify that no two transactions share the same `idempotency_key`.

- **Brute Force ($O(N^2)$):** Comparing each transaction against every other transaction requires: $(100,000 * 99,999) / 2 \approx 5 * 10^9$ operations $\rightarrow$ 5 to 10 seconds (CPU freeze and timeout).
- **Hash Set Optimization ($O(N)$ Time, $O(N)$ Space):** Checking existence in a hash set takes $O(1)$ on average: 100,000 lookups $\approx$ 0.015 seconds (15 milliseconds).
- **In-place Sort + Adjacent Scan ($O(N \log N)$ Time, $O(1)$ Auxiliary Space):** Useful when memory is constrained on embedded gateway appliances.

6. Code: Python Dynamic Array Resizing Proof

```
import sys
import time

def demonstrate_amortized_growth():
    """Demonstrates list capacity jumps in Python (CPython dynamic array)."""
    data = []
    prev_size = sys.getsizeof(data)
    print(f"{'Length':<10} | {'Byte Size':<12} | {'New Allocation Triggered?'}")
    print("-" * 50)

    for i in range(25):
        data.append(i)
        current_size = sys.getsizeof(data)
        resized = current_size != prev_size
        if resized:
            print(f"{'len(data)':<10} | {'current_size':<12} | YES (Grown to accommodate elements)")
            prev_size = current_size

if __name__ == "__main__":
    demonstrate_amortized_growth()
```

7. How It Works Internally

CPython & V8 Dynamic Array Growth Mechanics

1. In CPython, `list` is implemented as an array of pointers (`PyObject**`). When the allocated capacity is exhausted, CPython over-allocates using the formula: `new_allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`. This provides an approximate geometric growth factor of $\approx 1.125\times$.
2. In V8 (Node.js/JavaScript), `JSArray` has two modes: **Fast Elements** (contiguous C++ pointer backing stores) and **Dictionary Elements** (hash tables for sparse arrays). For fast elements, resizing multiplies capacity by $1.5\times$ to $2\times$.
3. Copying N elements takes $O(N)$, but because reallocation happens geometrically, resizing from 1 to N requires: $1 + 2 + 4 + 8 + \dots + N = 2N - 1 = O(N)$ total copies across N insertions. Total Time / Operations $= O(N) / N = O(1)$ amortized per append.

8. Common Mistakes

1. **Confusing Auxiliary Space with Total Space:** Auxiliary space is temporary working memory *excluding* the input. In MergeSort, total space is $O(N)$, and auxiliary space is $O(N)$. In QuickSort, auxiliary stack space is $O(\log N)$ average, $O(N)$ worst-case.
2. **Ignoring String Immutability Overhead:** In Python and JavaScript, strings are immutable. Doing `s += char` inside an N -iteration loop creates a new string each time, turning an intended $O(N)$ operation into $O(N^2)$! Always use `"".join(list)` in Python or an array push + `.join("")` in JS.
3. **Assuming Hash Table Lookups are Always $O(1)$:** If hash collisions occur (or an adversary triggers hash collision attacks), open-addressed or chained tables degrade to $O(N)$ worst case.

9. Performance / Complexity Matrix

Operation / Algorithm	Best Time	Average Time	Worst Time	Auxiliary Space
Array Access by Index	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Dynamic Array Append	$O(1)$	$O(1)$	$O(N)$	$O(1)$
Hash Table Insert/Lookup	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$ avg
Breadth-First Search (BFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Depth-First Search (DFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$ stack

10. Interview Questions (Easy → Medium → Hard) with Complete Model Answers

● Easy Question

Q: What is the difference between $O(1)$ and $O(\log N)$? Give an example of each in software engineering.

Complete Answer: - **$O(1)$ [Constant Time]:** The execution time of the operation is completely independent of the input size N . No matter if the dataset contains 10 items or 10 billion items, the operation finishes in the same fixed number of machine instructions. - *Example:* Accessing an array element by index (`arr[i]`), retrieving a key from an in-memory Hash Map / Redis cache on average, or popping an element from a stack. - **$O(\log N)$ [Logarithmic Time]:** The execution time grows in proportion to the logarithm of the input size. At every step, the algorithm cuts the remaining search

space by a constant fraction (usually half). Even for $N = 1,000,000,000$ (1 billion items), $\log_2(10^9) \approx 30$ operations! – *Example:* Binary Search on a sorted array, looking up an account number in a balanced Binary Search Tree (BST) or B+ Tree index in PostgreSQL/Oracle, or finding an element in a binary heap.

● Medium Question

Q: Explain why QuickSort is commonly preferred over MergeSort for arrays in cache-sensitive systems despite QuickSort having a worse worst-case time complexity ($O(N^2)$ vs $O(N \log N)$).

Complete Answer: There are three fundamental architectural reasons: 1. **Auxiliary Memory Allocation:** – MergeSort requires an auxiliary buffer of $O(N)$ extra memory because merging two sorted subarrays cannot easily be done in-place without significant performance penalties. – QuickSort operates strictly **in-place** with only $O(\log N)$ auxiliary call-stack memory. 2. **CPU Cache Locality & Hardware Prefetching:** – QuickSort's partitioning traverses memory **sequentially and contiguously** from both ends inward (`left` and `right` pointers). This exhibits exceptional **spatial locality of reference**. – Modern CPUs load entire 64-byte cache lines into L1/L2 caches. Because QuickSort accesses adjacent bytes, almost every read is an ultra-fast L1 cache hit (~1–2 ns) rather than a slow main RAM fetch (~60–100 ns). – MergeSort frequently copies elements back and forth between the primary array and auxiliary buffer, saturating CPU memory bus bandwidth and thrashing cache lines. 3. **Randomized / Dual-Pivot Mitigations:** – Modern implementations (such as Java's `Arrays.sort` for primitives) use **Dual-Pivot QuickSort** with randomized or median-of-three pivot selection. This makes the probability of encountering the pathological $O(N^2)$ worst-case virtually zero (1 in 2^{32}).

When is MergeSort preferred? – When **Stability** is non-negotiable (preserving the relative order of duplicate elements). – When sorting linked lists (where nodes are linked by pointers and no contiguous memory copying is required). – When sorting external files stored on disk that do not fit into RAM (External Merge Sort).

● Hard Question

Q: An algorithm processes a stream of financial transactions. Each incoming item requires checking a min-heap of size K and occasionally rebuilding an auxiliary index of size M . Derive the tight asymptotic bound for N total transactions where $M = \sqrt{N}$ and $K = \log N$.

Complete Mathematical Derivation: Let us break down the total work into two components: 1. **Heap Operation Cost (per transaction):** – For each of the N transactions, inserting/checking a min-heap of size K takes $O(\log K)$ time. – Given $K = \log N$: `Cost_heap =  $O(\log(\log N))$` per transaction. – Total heap cost across all N transactions: `Total_heap =  $N * O(\log(\log N)) = O(N \log(\log N))$` .

1. Index Rebuilding Cost:

2. An auxiliary index of size $M = \sqrt{N}$ is rebuilt periodically.

3. Rebuilding an index of size M takes `$O(M \log M)$` time (e.g., sorting or constructing a balanced tree).

4. If the index is rebuilt every M transactions, the total number of rebuilds across N transactions is:

$$\text{Number of Rebuilds} = N / M = N / \sqrt{N} = \sqrt{N}.$$

5. Work per rebuild: `$M \log M = \sqrt{N} * \log(\sqrt{N}) = \sqrt{N} * (1/2 * \log N) = (1/2) * \sqrt{N} \log N$` .

6. Total rebuild work across all $\sqrt{N}$ rebuilds: $\text{Total_rebuild} = \sqrt{N} * [(1/2) * \sqrt{N} \log N] = (1/2) * (\sqrt{N} * \sqrt{N}) \log N = (1/2) * N \log N = O(N \log N)$.

7. Combined Tight Bound:

8. $\text{Total Time} = \text{Total_heap} + \text{Total_rebuild} = O(N \log(\log N)) + O(N \log N)$.

9. Since $N \log N$ strictly dominates $N \log(\log N)$ as N grows large: $\text{Final Tight Bound} = \theta(N \log N)$.

11. Follow-up Questions from Interviewer with Complete Answers

Follow-up 1

Interviewer: "What is cache locality, and how does it affect the empirical execution speed of an $O(N)$ array traversal versus an $O(N)$ linked list traversal?"

Complete Answer: - **Cache Locality Principles:** - **Spatial Locality:** If a memory location is accessed, nearby memory locations are likely to be accessed soon. - **Temporal Locality:** If a memory location is accessed, the same location is likely to be accessed again in the near future. - **Why Arrays Beat Linked Lists by 10x–50x Empirically:** - An **array** is allocated in a single, contiguous block of virtual memory. When the CPU reads `arr[0]`, the hardware prefetcher automatically pulls the entire 64-byte CPU cache line (holding the next 8 to 16 integer elements) into L1 cache. Subsequent iterations (`arr[1]`, `arr[2]`, etc.) are instant L1 hits. - In a **linked list**, each node is allocated independently on the heap via dynamic memory allocators (`malloc` / garbage collector). Nodes are scattered unpredictably across RAM. - To traverse a linked list, the CPU must dereference `node.next` (known as **pointer chasing**). Because the next node is at an arbitrary memory address, the CPU experiences a **cache miss**, stalling the execution pipeline for 100–200 clock cycles while waiting for data from main memory. - **Conclusion:** While both algorithms are asymptotically $O(N)$, the array traversal completes in a fraction of the wall-clock time due to spatial cache locality and hardware prefetching.

Follow-up 2

Interviewer: "If `Array.prototype.sort()` in Node.js uses Timsort, what is its worst-case space complexity, and why doesn't it use standard in-place QuickSort?"

Complete Answer: 1. **Worst-Case Space Complexity of Timsort:** - Timsort requires **$O(N)$** auxiliary space in the worst case (or $O(N/2)$ buffer memory) to merge runs of sorted elements, alongside a small stack of $O(\log N)$ run descriptors. 2. **Why Node.js (V8) Replaced QuickSort with Timsort (starting in V8 v7.0):** - **Stability is Essential for Web Applications:** QuickSort is unstable (equal elements can change their relative order). For example, if a user sorts a banking transaction table by "Transaction Date" and then sorts by "Amount", an unstable sort scrambles the date ordering of equal-amount transactions! Timsort guarantees strict **stability**. - **Real-World Data is Already Partially Sorted:** Timsort is an adaptive algorithm derived from MergeSort and InsertionSort. On already sorted or reversed data (very common in logging and audit trails), Timsort detects natural runs and runs in **$O(N)$ best-case linear time!** QuickSort would still take $O(N \log N)$ or degrade to $O(N^2)$ without complex pivot balancing. -

Guaranteed $O(N \log N)$ Worst-Case: Unlike naive QuickSort which can degrade to $O(N^2)$ when encountering adversarial worst-case inputs (causing Node.js event loop freezes), Timsort strictly guarantees $O(N \log N)$ worst-case time.

Follow-up 3

Interviewer: "How does Python's list over-allocation strategy affect memory footprint when scaling to millions of ledger rows?"

Complete Answer:

- Pointer Indirection Overhead:** - In Python (CPython), a `list` does not store primitive numbers directly. It is an array of 8-byte C pointers (`PyObject*`), where each pointer references an independent heap object (`PyLongObject`). - A 64-bit Python integer object contains: - Reference count: 8 bytes - Type pointer (`ob_type`): 8 bytes - Size field: 8 bytes - Digit array: 4-8 bytes - **Total per integer object:** 28 bytes! - Plus the 8-byte pointer in the list = **36 bytes per integer element**.
- Geometric Over-Allocation:** - When elements are appended, CPython allocates extra unused slots: `new_allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`. - This means at any point, a list may hold 12.5% more allocated capacity than its current `len()`.
- Scale Comparison (10 Million Transactions):** - Raw Python List of integers: `10,000,000 * 36 bytes ≈ 360 MB`. - NumPy Array (`np.int64`) or C array: `10,000,000 * 8 bytes ≈ 80 MB` (contiguous, zero pointer overhead, 4.5x memory reduction).

Production Recommendation: For financial transaction processing pipelines exceeding 100,000 records, always migrate from raw Python lists to NumPy arrays, pandas DataFrames, or PyArrow tables to prevent memory exhaustion and garbage collection thrashing.

Follow-up 4

Interviewer: "How do you analyze the amortized cost of an append operation using both the Aggregate Method and the Accounting / Potential Method?"

Complete Answer:

1. The Aggregate Method

- In a sequence of N append operations starting from an empty array of capacity 1:
- Resizing occurs at sizes: 1, 2, 4, 8, 16, ..., 2^k (where $2^k \leq N$).
- The actual cost of insertion without resize is 1 for each of the N operations $\rightarrow$ Total = N .
- The copying cost during resizes is: $1 + 2 + 4 + 8 + \dots + 2^k < 2 * 2^k \leq 2N$.
- Total Cost across N operations = N (insertions) + $2N$ (copies) = $3N$.
- **Amortized cost per operation:** Total Cost / $N = 3N / N = 3 = O(1)$.

2. The Accounting Method (Token / Banker's Method)

- We assign an artificial **amortized charge** of 3 credits to each incoming append operation:
- **Credit 1:** Pays for the actual insertion of the new element into the current slot.
- **Credit 2:** Stored as savings on this new element to pay for its own future relocation.
- **Credit 3:** Stored as savings on an older element that has already moved once but needs to move again.

- When the array doubles from size K to $2K$, exactly K new elements have arrived since the last resize.
- Accumulated credit = $K * 2 = 2K$ credits!
- This accumulated credit is precisely enough to copy all $2K$ elements into the newly allocated memory buffer without charging any extra cost.
- Because the credit balance never drops below zero, the amortized cost is strictly $O(1)$.

3. The Potential Method (Physicist's Method)

- Define a potential function $\Phi(D_i)$ on the data structure state after step i : $\Phi(D_i) = 2 * \text{size}_i - \text{capacity}_i$
- Immediately after a doubling resize: $\text{size} = \text{capacity} / 2 \rightarrow \Phi = 2 * (\text{capacity} / 2) - \text{capacity} = 0$.
- Immediately before a doubling resize: $\text{size} = \text{capacity} \rightarrow \Phi = 2 * \text{capacity} - \text{capacity} = \text{capacity}$.
- Since $\Phi(D_i) \geq 0$ for all i , the potential is always non-negative.
- The amortized cost is defined as: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.
- **Case 1: No resize ($c_i = 1$):** $\text{size}_i = \text{size}_{i-1} + 1$, $\text{capacity}_i = \text{capacity}_{i-1}$. $\Delta\Phi = (2(\text{size} + 1) - \text{capacity}) - (2 * \text{size} - \text{capacity}) = 2$. $\hat{c}_i = 1 + 2 = 3$.
- **Case 2: Resize occurs ($c_i = \text{size}_{i-1} + 1$):** $\text{capacity}_i = 2 * \text{capacity}_{i-1} = 2 * \text{size}_{i-1}$. $\Phi(D_{i-1}) = \text{size}_{i-1}$. $\Phi(D_i) = 2 * (\text{size}_{i-1} + 1) - 2 * \text{size}_{i-1} = 2$. $\Delta\Phi = 2 - \text{size}_{i-1}$. $\hat{c}_i = (\text{size}_{i-1} + 1) + (2 - \text{size}_{i-1}) = 3$.
- In both cases, the amortized cost is strictly $3 = O(1)$.

12. Practical Exercise

Analyze the time and auxiliary space complexity of each function:

```
# Exercise 1:
def mystery_one(n: int):
    count = 0
    i = n
    while i > 0:
        for j in range(i):
            count += 1
        i /= 2
    return count

# Exercise 2:
def mystery_two(arr: list[int]):
    n = len(arr)
    result = []
    for i in range(n):
        subset = []
        for j in range(i, n):
            subset.append(arr[j])
            result.append(list(subset))
    return result
```

Complete Mathematical Solutions:

Solution for Mystery 1:

- The outer loop divides `i` by 2 at each iteration (`n, n/2, n/4, n/8, ..., 1`).
- The inner loop executes `i` times.
- Total iterations: $T(N) = N + N/2 + N/4 + N/8 + \dots + 1$ $T(N) = N * (1 + 1/2 + 1/4 + 1/8 + \dots)$ This is a converging geometric series with sum $= N * [1 / (1 - 1/2)] = 2N$.
- **Time Complexity: $O(N)$** (Linear, NOT $O(N \log N)$!).
- **Auxiliary Space Complexity: $O(1)$** (Only scalar counter variables).

Solution for Mystery 2:

- The outer loop runs N times. The inner loop runs from `i` to N.
- Total iterations $= N + (N - 1) + (N - 2) + \dots + 1 = N(N + 1) / 2 = O(N^2)$.
- However, inside the loop, `result.append(list(subset))` creates a shallow copy of the current `subset`.
- The average length of `subset` is $O(N)$. Copying an array of length K takes $O(K)$ time!
- Total time $= \Sigma(\text{iterations}) * \text{copy_length} = O(N^2 * N) = O(N^3)$.
- **Time Complexity: $O(N^3)$** .
- **Auxiliary Space Complexity: $O(N^3)$** (to store all generated subarrays in the result list).

13. Quick Revision

- Big-O denotes worst-case upper bound; Big- Θ denotes tight asymptotic bound; Big- Ω denotes lower bound.
- Geometric series: $N + N/2 + N/4 + \dots = 2N = O(N)$.
- In-place algorithms utilize $O(1)$ auxiliary space beyond input buffers.
- String concatenation inside loops in Python/JS creates $O(N^2)$ memory churning due to immutability.
- Dynamic array append is $O(1)$ amortized, $O(N)$ worst-case.

14. Interview Checklist

- [x] Can define Big-O mathematically ($0 \leq f(N) \leq c * g(N)$).
- [x] Can explain the accounting/potential method for amortized analysis.
- [x] Can distinguish auxiliary space from input space.
- [x] Can explain CPU cache lines and spatial/temporal locality.
- [x] Can identify hidden $O(N)$ costs in high-level language methods (e.g., `list.insert(0, val)` or `Array.shift()` which shifts all elements).
- [x] Can mathematically derive complex series summation bounds on whiteboard.

 CHAPTER 01 SUMMARY & REVISION CHECKLIST **WHAT TO REMEMBER**

- Basic algebra (logarithms, exponents, summation formulas: $\sum_{i=1}^n i = n(n+1)/2$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).
- Big-O (O): Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $0 \leq f(N)$

 CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

Python Fundamentals, OOP, Internals & Advanced Concepts

PRIORITY

 High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Python Fundamentals, OOP, Internals & Advanced Concepts

1. Why This Matters (Interview Perspective)

IDFC FIRST Bank ke job description mein "**Java / Python / C++**" explicitly mentioned hai. Aapka primary backend experience Node.js/Express mein hai aur aapne Python mainly DSA ke liye use kiya hai. Lekin interview mein agar interviewer Python par switch karta hai, toh woh sirf basic loops nahi puchega. Woh puchega: – Python ka memory model (pointers, heap allocations, mutability). – Generators (`yield`) vs Lists memory footprint. – Decorators kaise kaam karte hain under the hood. – Magic methods (`__eq__` , `__repr__` , `__lt__`) custom sorting aur OOP design ke liye. Is module ka goal aapko data scientist banana nahi hai, balki **ek confident 3+ YoE developer ki tarah Python discuss aur code karne ke kabil banana hai**.

2. Python Fundamentals (Hinglish + Code)

Simple Language Mein:

 KEY ENGINEERING INSIGHT

"Python mein sab kuch ek **Object** hota hai. Variables koi memory box nahi hote jisme value rakhi hoti hai; variables bas ek **Name Tag (Pointer)** hote hain jo heap par bane object ko point karte hain."

1. Variables, Data Types & Slicing

```
# Numbers & Strings
age: int = 28
balance: float = 45200.50
bank_name: str = "IDFC FIRST Bank"

# Slicing: [start : stop : step]
account_num = "IDFB1000928371"
bank_code = account_num[0:4] # 'IDFB'
last_four = account_num[-4:] # '8371'
reversed_str = account_num[::-1] # Pure string reverse in O(N)
```

2. Lists, Tuples, Sets, Dictionaries

Data Structure	Syntax	Mutability	Duplicates?	Ordered?	Under the Hood
<code>list</code>	<code>[1, 2, 3]</code>	Mutable	Yes	Yes	Dynamic Array (<code>PyObject**</code>)
<code>tuple</code>	<code>(1, 2, 3)</code>	Immutable	Yes	Yes	Fixed-size array (Cache friendly)
<code>set</code>	<code>{1, 2, 3}</code>	Mutable	No	No	Hash Table (Keys only, O(1) avg)
<code>dict</code>	<code>{"a": 1}</code>	Mutable	No (Keys)	Yes (3.7+)	Compact Hash Table (O(1) avg)

```
# Unpacking and *args, **kwargs
def process_transaction(txn_id: str, *amounts, **metadata):
    """
    *amounts captures positional args as a tuple
    **metadata captures keyword args as a dictionary
    """
    total = sum(amounts)
    channel = metadata.get("channel", "UPI")
    print(f"Txn: {txn_id}, Total: ₹{total}, Channel: {channel}")

process_transaction("TXN_101", 500, 1200, 350, channel="IMPS", merchant="Swiggy")
# Output: Txn: TXN_101, Total: ₹2050, Channel: IMPS
```

3. Comprehensions (List, Dict, Set)

KEY ENGINEERING INSIGHT

"Loop likhne ke bajaye ek line mein readable aur fast collection banana."

```
# List Comprehension (Filtered & Transformed)
raw_amounts = [1500, -200, 8000, 0, 12000]
credits_only = [amt for amt in raw_amounts if amt > 0]
# [1500, 8000, 12000]

# Dict Comprehension: Customer ID to normalized account map
accounts = [("CUST_101", "SAVINGS"), ("CUST_102", "CURRENT")]
acc_dict = {cust_id: acc_type for cust_id, acc_type in accounts}
# {'CUST_101': 'SAVINGS', 'CUST_102': 'CURRENT'}

# Set Comprehension: Unique merchant categories
merchants = ["Swiggy", "Zomato", "Swiggy", "Amazon", "Uber"]
unique_merchants = {m.upper() for m in merchants}
# {'SWIGGY', 'ZOMATO', 'AMAZON', 'UBER'}
```

3. Python OOP (Object-Oriented Programming)

Core Pillars:

1. **Encapsulation:** Data aur methods ko ek unit (class) mein bind karna. Private members convention se `_single_underscore` (internal) ya `__double_underscore` (name mangling) hote hain.

2. **Inheritance:** Parent class ki functionality reuse karna.
3. **Polymorphism:** Same method name different classes mein alag behave kare (Duck Typing: *"If it walks like a duck and quacks like a duck, it's a duck"*).
4. **Abstraction:** Complex implementation hide karke clean interface provide karna (`abc.ABC`).

```
from abc import ABC, abstractmethod

# Abstraction: Interface for payment rails
class PaymentRail(ABC):
    @abstractmethod
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        pass

# Inheritance & Polymorphism
class UpiRail(PaymentRail):
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        print(f"[UPI] Transferring ₹{amount} from {from_acc} to {to_acc} via NPCI")
        return True

class NeftRail(PaymentRail):
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        print(f"[NEFT] Queuing ₹{amount} in RBI half-hourly clearing batch")
        return True
```

Magic Methods (Dunder Methods)

Interviewers love asking about `__repr__`, `__eq__`, aur `__lt__` :

```
class BankAccount:
    def __init__(self, account_number: str, balance: float):
        self.account_number = account_number
        self.balance = balance

    # User-friendly string (used by print(acc) and str(acc))
    def __str__(self) → str:
        return f"Account({self.account_number}): ₹{self.balance:,.2f}"

    # Developer/Debug representation (used in interactive shells and logs)
    def __repr__(self) → str:
        return f"BankAccount(account_number='{self.account_number}', balance={self.balance})"

    # Equality check (acc1 == acc2)
    def __eq__(self, other: object) → bool:
        if not isinstance(other, BankAccount):
            return False
        return self.account_number == other.account_number

    # Less-than comparator (Unlocks natural sorting: accounts.sort() by balance!)
    def __lt__(self, other: "BankAccount") → bool:
        return self.balance < other.balance

# Testing Dunder methods:
acc1 = BankAccount("IDFC_101", 45000.0)
acc2 = BankAccount("IDFC_102", 12000.0)
acc3 = BankAccount("IDFC_103", 95000.0)

accounts = [acc1, acc2, acc3]
accounts.sort() # Uses __lt__ automatically!
print(accounts)
# Output sorted by balance: [IDFC_102 (12k), IDFC_101 (45k), IDFC_103 (95k)]
```

4. Python Internals (Memory, GC, Copying & Generators)

1. References & Mutability

KEY ENGINEERING INSIGHT

"Numbers, Strings, Tuples immutable hote hain (unhe modify nahi kiya ja sakta, naya object banta hai). Lists, Dicts, Sets mutable hote hain (same memory address par modify hote hain)."

```
# Pass-by-object-reference trap:
def add_bonus(acc_balance: float, transaction_list: list):
    acc_balance += 500          # Float is immutable → local variable rebinds!
    transaction_list.append(500) # List is mutable → caller's list updates in place!

bal = 1000.0
txns = []
add_bonus(bal, txns)
print(bal)    # Still 1000.0 (Unchanged!)
print(txns)   # [500] (Mutated!)
```

2. Shallow Copy vs Deep Copy

- **Shallow Copy** (`copy.copy()` or `list.copy()`): Outer container ka naya object banta hai, lekin nested objects ke references same rehte hain.
- **Deep Copy** (`copy.deepcopy()`): Recursively saare nested objects ki independent duplicate copy banata hai.

```
import copy

nested = [[1, 2], [3, 4]]
shallow = copy.copy(nested)
deep = copy.deepcopy(nested)

nested[0][0] = 999
print(shallow[0][0]) # 999 (Affected! Shared nested reference)
print(deep[0][0])    # 1   (Unaffected! Fully independent clone)
```

3. Garbage Collection & Memory Management

Python memory management do mechanisms par chalti hai: 1. **Reference Counting (Primary)**: Har object ke header mein `ob_refcnt` hota hai. Jab koi variable us object ko point karta hai, count badhta hai (`+1`). Jab variable scope se bahar nikalta hai ya `del` hota hai, count ghat-ta hai (`-1`). Jaise hi `ob_refcnt == 0`, memory **immediately free** ho jaati hai. 2. **Generational Garbage Collector (Cyclic GC)**: - Agar do objects ek doosre ko refer kar rahe hain (`A.child = B` aur `B.parent = A`), toh dono ka reference count kabhi zero nahi hoga (**Reference Cycle**). - Python ka cyclic GC 3 generations (Gen 0, Gen 1, Gen 2) maintain karta hai. Young objects Gen 0 mein aate hain. GC periodic interval par circular references detect karke collect karta hai.

4. Generators (`yield`) — Processing Millions of Records Without OOM

KEY ENGINEERING INSIGHT

"Agar 10 million transactions ka CSV read karna hai, toh list mein read karne se 8GB RAM bhar jayegi aur server crash ho jayega. Generator ek time par **sirf ek item** memory mein rakhta hai."

```
from typing import Generator

def read_large_transaction_stream(file_path: str) → Generator[dict, None, None]:
    """Generator: Produces one record at a time in O(1) memory."""
    with open(file_path, "r") as f:
        header = f.readline().strip().split(",")
        for line in f:
            values = line.strip().split(",")
            yield dict(zip(header, values)) # Pauses execution and yields control!

# Usage:
# stream = read_large_transaction_stream("huge_txns.csv")
# for txn in stream:
#     process_payment(txn) # Memory stays strictly bounded under 20MB!
```

5. Decorators — Enterprise Cross-Cutting Concerns (Logging, Auth, Timing)

KEY ENGINEERING INSIGHT

"Decorator ek function hota hai jo doosre function ko argument ki tarah leta hai, uski functionality extend karta hai, aur naya function return karta hai."

```
import time
from functools import wraps

def audit_log(func):
    """Decorator that logs execution time and parameters for banking audits."""
    @wraps(func) # Preserves original function's name and docstring
    def wrapper(*args, **kwargs):
        start_time = time.time()
        print(f"[AUDIT] Starting {func.__name__} with args={args}")
        try:
            result = func(*args, **kwargs)
            duration_ms = (time.time() - start_time) * 1000
            print(f"[AUDIT] {func.__name__} completed in {duration_ms:.2f}ms")
            return result
        except Exception as e:
            print(f"[AUDIT ALERT] {func.__name__} failed with error: {e}")
            raise
    return wrapper

@audit_log
def debit_account(account_id: str, amount: float):
    time.sleep(0.05) # Simulate DB latency
    return f"Debited ₹{amount} from {account_id}"

print(debit_account("ACC_1001", 2500.0))
```

6. Context Managers (`with` statement)

KEY ENGINEERING INSIGHT

"Resource cleanup (DB connections close karna, files close karna, locks release karna) guarantee karta hai, chahe exception hi kyu na aa jaye."

```
class DatabaseTransaction:
    """Custom Context Manager simulating ACID transaction scope."""
    def __init__(self, connection_name: str):
        self.conn = connection_name

    def __enter__(self):
        print(f"BEGIN TRANSACTION on {self.conn}")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"ROLLBACK TRANSACTION due to exception: {exc_val}")
            return False # Re-raise exception
        else:
            print("COMMIT TRANSACTION")
            return True

# Safe usage:
with DatabaseTransaction("IDFC_Core_DB"):
    print("Executing SQL: UPDATE accounts SET balance = balance - 1000")
    # If error happens here, ROLLBACK triggers automatically!
```

5. Advanced Python Tools (`collections` , `itertools` , `dataclasses`)

```
# 1. dataclasses: Clean data containers without boilerplate __init__ and __repr__
from dataclasses import dataclass

@dataclass
class CustomerProfile:
    customer_id: str
    pan_number: str
    kyc_verified: bool
    risk_score: float = 0.0

cust = CustomerProfile("CUST_101", "ABCDE1234F", True)
print(cust) # CustomerProfile(customer_id='CUST_101', pan_number='ABCDE1234F',
kyc_verified=True, risk_score=0.0)

# 2. functools.lru_cache: Built-in Memoization
from functools import lru_cache

@lru_cache(maxsize=128)
def calculate_compound_interest(principal: float, rate: float, years: int) → float:
    print(f"Computing interest for P={principal}, r={rate}, t={years}")
    return principal * ((1 + rate) ** years)

# First call computes; second call returns from cache in O(1)!
calculate_compound_interest(100000, 0.07, 5)
calculate_compound_interest(100000, 0.07, 5) # Instant cache hit!
```

6. Quick Revision & Interview Flashcards

- **Variables = Pointers:** Python variables heap objects ko refer karte hain.
- **Reference Counting + Generational GC:** Python ref count 0 hote hi free karta hai; cycles ko cyclic GC handle karta hai.
- **Generators (`yield`):** Lazy evaluation, memory consumption strictly O(1).
- **Decorators:** Functions that wrap other functions (syntactic sugar `@decorator`).
- **Context Managers (`__enter__` , `__exit__`):** Guarantees resource cleanup even on crashes.



CHAPTER 02 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Python ka memory model (pointers, heap allocations, mutability).
- Generators (`yield`) vs Lists memory footprint.
- Decorators kaise kaam karte hain under the hood.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

NumPy: Numerical Computing & Vectorized Operations

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

1 2 3 4 NumPy: Numerical Computing & Vectorized Operations

1. What is NumPy & Why Does It Exist?

Simple Language Mein:

🔑 30–SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

"Python ke normal `list` mein agar hum 10 million numbers ka sum karein ya unhe 2 se multiply karein, toh Python ka loop bohot slow chalta hai kyunki Python har element ke liye type checking karta hai aur memory mein pointers follow karta hai.

NumPy (Numerical Python) C-language mein likhi hui library hai jo numerical data ko **contiguous block of memory** mein store karti hai aur CPU ke hardware instructions (**SIMD – Single Instruction Multiple Data**) use karke ek saath hazaron calculations karti hai bina kisi Python loop ke. Isi ko **Vectorization** kehte hain."

2. NumPy ndarray vs Python List

```
flowchart TD
    subgraph PyList["Python List: Array of Pointers (Scattered Heap Memory)"]
        L0["Pointer 0"] --> O0["PyObject: Int 10\n(Header, RefCount, Type, Value)"]
        L1["Pointer 1"] --> O1["PyObject: Int 20\n(Header, RefCount, Type, Value)"]
        L2["Pointer 2"] --> O2["PyObject: Int 30\n(Header, RefCount, Type, Value)"]
    end

    subgraph NumPyArray["NumPy ndarray: Contiguous C-Style Buffer in RAM"]
        N["[ int64: 10 ][ int64: 20 ][ int64: 30 ] (Raw 8-byte integers in sequential memory)"]
    end
```

Feature	Python Standard <code>list</code>	NumPy <code>ndarray</code>
Memory Layout	Pointers to individual heap objects	Contiguous raw memory block
Data Types	Heterogeneous (can mix <code>int</code> , <code>str</code> , <code>dict</code>)	Homogeneous (all elements share exact same <code>dtype</code>)
Performance	Slow (Type checking & pointer dereferencing)	Blazing fast (Compiled C code + CPU SIMD vectorization)
Memory Size	High overhead (~28 bytes per integer)	Minimal (8 bytes for <code>int64</code> , 4 bytes for <code>float32</code>)
Operations	<code>list * 2</code> duplicates list (<code>[1, 2] * 2 = [1, 2, 1, 2]</code>)	<code>arr * 2</code> multiplies every element (<code>[2, 4]</code>)

3. Core Concepts: Dimensions, Shape, Size, Dtype & Axis

```
import numpy as np

# 2D Array: Shape (2, 3) → 2 rows, 3 columns
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
], dtype=np.float64)

print("Shape:", matrix.shape) # (2, 3)
print("Dimensions:", matrix.ndim) # 2 (2D array)
print("Total Elements (Size):", matrix.size) # 6
print("Data Type:", matrix.dtype) # float64
```

The "Axis" Concept Demystified:

 **KEY ENGINEERING INSIGHT**

"2D array mein: - `axis=0` hamesha **Rows ke across vertical direction** mein operate karta hai (Column-wise result deta hai). - `axis=1` hamesha **Columns ke across horizontal direction** mein operate karta hai (Row-wise result deta hai)."

```
# Sum along axis=0 (Down the columns):
print(np.sum(matrix, axis=0)) # [10+40, 20+50, 30+60] → [50., 70., 90.]

# Sum along axis=1 (Across the rows):
print(np.sum(matrix, axis=1)) # [10+20+30, 40+50+60] → [60., 150.]
```

4. Creating NumPy Arrays

```
# 1. Direct from list
a = np.array([1, 2, 3])

# 2. Zeros and Ones (Allocation without garbage data)
zeros = np.zeros((3, 4)) # 3 rows, 4 columns of 0.0
ones = np.ones((2, 3), dtype=np.int32) # 2x3 matrix of 1s

# 3. arange: Like range() but returns ndarray (start, stop, step)
seq = np.arange(10, 50, 5) # [10, 15, 20, 25, 30, 35, 40, 45]

# 4. linspace: Linear spacing (start, stop, num_points)
# Great for financial curves and probability distributions
curve = np.linspace(0.0, 1.0, 5) # [0.0, 0.25, 0.5, 0.75, 1.0]

# 5. Identity Matrix: Diagonal 1s, rest 0s
eye_matrix = np.eye(3) # 3x3 identity matrix
```

5. Array Operations: Reshaping, Slicing & Views vs Copies

Slicing & The "View" Trap:

KEY ENGINEERING INSIGHT

"NumPy mein slice lene par data copy nahi hota; woh original array ka **View (Window)** hota hai. Agar view modify karoge, toh original array bhi change ho jayega!"

```
arr = np.array([10, 20, 30, 40, 50])
view_slice = arr[1:4] # Elements: [20, 30, 40]
view_slice[0] = 999 # Modifying the view!
print(arr) # [10, 999, 30, 40, 50] → ORIGINAL ARRAY MUTATED!

# Explicit Copy (Safe):
safe_copy = arr[1:4].copy()
safe_copy[0] = 777
print(arr[1]) # 999 (Original remains untouched)
```

Reshaping & Flattening:

```
orig = np.arange(12) # [0, 1, 2, ... 11]

# Reshape to 3x4 matrix
reshaped = orig.reshape(3, 4)

# Reshape with -1 (NumPy automatically calculates the remaining dimension!)
auto_reshaped = orig.reshape(2, -1) # Shape becomes (2, 6)

# Flattening: flatten() vs ravel()
# flatten() hamesha naya copy banata hai
flat_copy = reshaped.flatten()

# ravel() view return karne ki koshish karta hai (zero memory copy!)
flat_view = reshaped.ravel()
```

6. Vectorization: Why It Beats Python Loops by 50x

Code Benchmark: Python Loop vs NumPy Vectorization

```
import time

size = 5_000_000
py_list = list(range(size))
np_arr = np.arange(size)

# Test 1: Python Loop
start = time.time()
py_result = [x * 2 for x in py_list]
py_time = time.time() - start

# Test 2: NumPy Vectorization
start = time.time()
np_result = np_arr * 2
np_time = time.time() - start

print(f"Python List Loop: {py_time:.4f} seconds")
print(f"NumPy Vectorized: {np_time:.4f} seconds")
print(f"Speedup: {py_time / np_time:.1f}x FASTER!")
# Output: NumPy is typically 40x to 80x faster!
```

Kyu fast hai? 1. C-level loop runs inside CPU registers without Python bytecode overhead. 2. Hardware prefetching: Contiguous bytes sequentially load into L1/L2 cache without cache misses. 3. SIMD (AVX-512): Single CPU instruction multiplies 8 floating-point numbers in a single clock cycle.

7. Broadcasting Rules (The Most Asked NumPy Concept)

Simple Language Mein:

KEY ENGINEERING INSIGHT

"Broadcasting ek aisa rule hai jiske through NumPy alag-alag shapes ke arrays par arithmetic operations perform karne deta hai bina memory mein extra copy banaye."

```
flowchart LR
    subgraph Operation["Broadcasting: (3, 3) + (1, 3)"]
        A["Matrix A: (3, 3)\n[[1, 2, 3],\n [4, 5, 6],\n [7, 8, 9]]"]
        B["Row B: (1, 3)\n[[10, 20, 30]]"]
        Result["Row B is conceptually stretched across 3 rows:\n[[11, 22, 33],\n [14, 25, 36],\n [17, 28, 39]]"]
    end
    A --> Result
    B --> Result
```

The Two Strict Rules of Broadcasting:

Piche se (from trailing dimensions to leading dimensions) compare karo: 1. Dono dimensions **equal** honi chahiye, **OR** 2. Kisi ek dimension ka size **1** hona chahiye. Agar dimension 1 hai, toh woh doosre array ke dimension se match karne ke liye virtual stretch (replicate) ho jata hai.

```
# Example: Adding a scalar (Shape: ()) to a 2D matrix (Shape: (2, 2))
mat = np.array([[1, 2], [3, 4]])
result = mat + 10 # 10 broadcasts to [[10, 10], [10, 10]]
# Result: [[11, 12], [13, 14]]
```

8. Boolean Indexing (Filtering Data)

```
# Banking Transaction Amounts:
transactions = np.array([250.0, 12000.0, 450.0, 95000.0, 3200.0, 78000.0])

# Step 1: Boolean mask create karo
mask = transactions > 50000.0
print(mask) # [False, False, False, True, False, True]

# Step 2: Mask se filter karo
suspicious_high_txns = transactions[mask]
print(suspicious_high_txns) # [95000., 78000.]

# Compound filtering (& for AND, | for OR, ~ for NOT):
flagged = transactions[(transactions >= 1000) & (transactions <= 80000)]
```

9. NumPy Interview Question Bank (40+ Questions)

Part A: 20 Conceptual Questions

1. NumPy array aur Python list mein 3 fundamental differences kya hain?

Answer: Contiguous vs pointer memory, homogeneous vs heterogeneous types, element-wise vectorized arithmetic vs repetition.

2. NumPy ka `ndarray` internal structure kaisa hota hai?

Answer: Ek data pointer (raw memory), `dtype`, `shape` tuple, aur `strides` tuple (bytes to step to reach next element along each dimension).

3. What is a Strides tuple in NumPy?

Answer: Har dimension mein agle element tak jump karne ke liye kitne bytes aage badhna hai.
Example: Shape `(2, 3)` ke `int64` array ke strides `(24, 8)` hote hain.

4. Difference between `np.copy()` and View?

5. What is Broadcasting and what are its two mathematical conditions?

6. What is Vectorization and why does it avoid the Global Interpreter Lock (GIL)?

7. Explain the difference between `axis=0` and `axis=1` in a 2D matrix.

8. What does `reshape(3, -1)` do?

9. Difference between `np.zeros()` and `np.empty()` ?

Answer: `zeros` memory ko 0 se initialize karta hai; `empty` uninitialized garbage RAM points return karta hai (slightly faster, but dangerous).

10. Difference between `arr.flatten()` and `arr.ravel()` ?

11. How does boolean indexing work under the hood?

12. Can a NumPy array contain multiple data types? What happens if you pass `[1, "IDFC", 3.5]` ?

Answer: Automatic type upcasting happens. Saare elements string (`<U21`) ban jayenge.

13. What is SIMD and how does NumPy utilize it?

14. How do you find the memory footprint of an array? (`arr.nbytes` or `arr.size * arr.itemsize`).

15. What is the difference between `np.dot()` and `*` operator on two 2D arrays?

Answer: `*` element-wise multiplication karta hai; `np.dot()` / `@` linear algebra matrix multiplication karta hai.

16. How does `np.nan` behave in comparisons (`np.nan == np.nan`)?

Answer: `False` ! Always use `np.isnan(val)`.

17. What is the purpose of `np.linspace()` versus `np.arange()` ?

18. What is Fancy Indexing in NumPy?

19. How do you replace values in an array conditioned on a boolean predicate? (`np.where(condition, x, y)`).

20. Why is `np.sum(arr)` faster than Python's built-in `sum(arr)` on a NumPy array?

Part B: 10 Coding Questions

- 1. Normalize Array:** Given a 1D array of account balances, scale them between 0 and 1: `(arr - arr.min()) / (arr.max() - arr.min())`.
- 2. Find Indices of Outliers:** Return indices of transactions that are 3 standard deviations away from the mean: `np.where(np.abs(arr - np.mean(arr)) > 3 * np.std(arr))[0]`.
- 3. Count Non-Zero Elements:** `np.count_nonzero(arr)`.
- 4. Extract Diagonal Elements:** `np.diag(matrix)`.
- 5. Clip Values (Upper and Lower bounds):** Restrict transaction amounts between ₹100 and ₹50,000: `np.clip(arr, 100, 50000)`.
- 6. Reverse Array In-Place:** `arr[::-1]` (view) or `np.flip(arr)`.
- 7. Find Most Frequent Value in Array:** `vals, counts = np.unique(arr, return_counts=True); vals[np.argmax(counts)]`.
- 8. Replace all NaN with Mean:** `arr[np.isnan(arr)] = np.nanmean(arr)`.
- 9. Matrix Multiplication:** `C = A @ B` or `np.matmul(A, B)`.
- 10. Compute Moving / Rolling Average using convolution:** `np.convolve(arr, np.ones(window)/window, mode='valid')`.

Part C: 10 Comparison Questions

1. `list.append()` vs `np.append()` ? (Note: `np.append` creates a brand new array every time; never use it in a loop!).
2. `np.concatenate` vs `np.vstack` vs `np.hstack` ?
3. In-place operation (`arr += 1`) vs new assignment (`arr = arr + 1`)?
4. `np.mean()` vs `np.nanmean()` ?
5. `np.argmax()` vs `np.max()` ?
6. Deep copy vs shallow view in multi-dimensional slicing?
7. Integer array indexing vs boolean mask indexing?
8. Memory footprint: 1,000,000 Python ints vs 1,000,000 `np.int32` ? (~28MB vs 4MB).
9. Memory order: C-contiguous (row-major) vs Fortran-contiguous (column-major)?
10. `np.ascontiguousarray()` kab use karte hain?

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of NumPy: Numerical Computing & Vectorized Operations
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

Complete pandas Mastery: Data School Q&A Curriculum (38 Videos)

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



Complete pandas Mastery: Data School Q&A Curriculum (38 Videos)



HINGLISH MENTAL MODEL (INTERVIEW PITCH)

Source Verification: Structured directly from Kevin Markham's (Data School) renowned 38–video YouTube series: *"Easier data analysis in Python with pandas"* ([Playlist](#) [PL5-da3q6B5ICCsgW1Mx1Z0Hq8LL5U3u9y](#)). Includes 100% topic coverage, banking/fintech production scenarios, Hinglish mental models, memory optimization, and IDFC FIRST Bank interview questions.

Master Traceability Matrix (Videos 1 to 38)

#	Video Title	Length	Concepts Covered	Curriculum Chapter
1	What is pandas?	6:24	Tabular data representation, 1D vs 2D, NumPy relationship	Chapter 1
2	How do I read a tabular data file into pandas?	8:54	<code>read_csv()</code> , <code>sep</code> , <code>header</code> , <code>names</code> , <code>skiprows</code> , <code>nrows</code>	Chapter 1
3	How do I select a pandas Series from a DataFrame?	11:10	Bracket vs dot notation, Series extraction, column concatenation	Chapter 1
4	Why do some pandas commands end with parentheses (and others don't)?	8:45	Methods (actions) vs Attributes (state metadata)	Chapter 1
5	How do I rename columns in a pandas DataFrame?	9:36	<code>rename(columns={...})</code> , <code>df.columns = [...]</code> , <code>str.replace()</code>	Chapter 2
6	How do I remove columns from a pandas DataFrame?	6:35	<code>df.drop()</code> , <code>axis=1</code> , multi-column drop, row drop (<code>axis=0</code>)	Chapter 2
7	How do I sort a pandas DataFrame or a Series?	8:56	<code>Series.sort_values()</code> , multi-column sort, ascending flags	Chapter 3
8	How do I filter rows of a pandas DataFrame by column value?	13:44	Boolean indexing, masking, comparison operators	Chapter 3
9	How do I apply multiple filter criteria to a pandas DataFrame?	9:51	Logical <code>&</code> , <code> </code> , parentheses requirement, <code>.isin()</code> operator	Chapter 3
10	Your pandas questions answered! (Part 1)	9:06	<code>usecols</code> , <code>nrows</code> , <code>iterrows</code> performance trap, <code>select_dtypes</code>	Chapter 3
11	How do I use the "axis" parameter in pandas?	8:33	<code>axis=0</code> (down rows) vs <code>axis=1</code> (across columns), row/col stats	Chapter 2
12	How do I use string methods in pandas?	6:16	<code>.str</code> accessor, <code>upper()</code> , <code>contains()</code> , <code>replace()</code> , regex	Chapter 4
13	How do I change the data type of a pandas Series?	7:28	<code>astype()</code> , currency cleaning, <code>pd.to_numeric(errors='coerce')</code>	Chapter 5
14	When should I use a "groupby" in pandas?	8:24	Split-Apply-Combine, aggregation functions, <code>.agg(['min', 'max'])</code>	Chapter 10

#	Video Title	Length	Concepts Covered	Curriculum Chapter
15	How do I explore a pandas Series?	9:50	<code>describe()</code> , <code>value_counts(normalize=True)</code> , <code>nunique()</code> , <code>crosstab</code>	Chapter 6
16	How do I handle missing values in pandas?	14:27	<code>isna()</code> , <code>dropna(how='any' 'all')</code> , <code>fillna()</code> , <code>bfill</code> / <code>ffill</code>	Chapter 7
17	What do I need to know about the pandas index? (Part 1)	13:36	Index as identifier, selection anchor, alignment in math	Chapter 8
18	What do I need to know about the pandas index? (Part 2)	10:38	Alignment mechanics (<code>s1 + s2</code>), <code>set_index()</code> , <code>reset_index()</code>	Chapter 8
19	How do I select multiple rows and columns from a pandas DataFrame?	21:46	<code>loc</code> (labels, inclusive) vs <code>iloc</code> (integers, exclusive), <code>ix</code> deprecation	Chapter 9
20	When should I use the "inplace" parameter in pandas?	10:18	<code>inplace=True</code> performance myth, chaining advantages	Chapter 9
21	How do I make my pandas DataFrame smaller and faster?	19:05	<code>info(memory_usage='deep')</code> , <code>category</code> dtype, integer codes	Chapter 11
22	How do I use pandas with scikit-learn to create Kaggle submissions?	13:25	Feature matrix <code>X</code> , target <code>y</code> , <code>.values</code> , test set alignment	Chapter 13
23	More of your pandas questions answered! (Part 2)	19:23	<code>df.sample()</code> , <code>Series.interpolate()</code> , train/test splits	Chapter 7 & 15
24	How do I create dummy variables in pandas?	13:13	<code>pd.get_dummies()</code> , <code>drop_first=True</code> , dummy variable trap	Chapter 13
25	How do I work with dates and times in pandas?	10:20	<code>pd.to_datetime()</code> , <code>.dt</code> accessor, date filtering, timedeltas	Chapter 12
26	How do I find and remove duplicate rows in pandas?	9:47	<code>uplicated(keep='first' 'last' False)</code> , <code>drop_duplicates()</code>	Chapter 13
27	How do I avoid a SettingWithCopyWarning in pandas?	13:29	Chained assignment vs <code>.loc</code> , Views vs Copies, <code>.copy()</code> fix	Chapter 9
28	How do I change display options in pandas?	14:55	<code>pd.set_option()</code> , <code>max_rows</code> , <code>max_columns</code> , <code>precision</code>	Chapter 14

#	Video Title	Length	Concepts Covered	Curriculum Chapter
29	How do I create a pandas DataFrame from another object?	14:25	Dict of lists, list of dicts, 2D NumPy array constructor	Chapter 14
30	How do I apply a function to a pandas Series or DataFrame?	17:57	<code>Series.map()</code> , <code>Series.apply()</code> , <code>df.apply(axis=0 1)</code> , <code>df.map()</code>	Chapter 14
31	Bonus: How do I use the MultiIndex in pandas?	25:00	Hierarchical indexing, <code>unstack()</code> , <code>stack()</code> , cross-section <code>.xs()</code>	Chapter 15
32	Bonus: How do I merge DataFrames in pandas?	21:48	<code>pd.merge()</code> , inner/left/right/outer, <code>indicator=True</code> , <code>validate</code>	Chapter 15
33	Bonus: 4 new time-saving tricks in pandas	14:50	<code>df.query()</code> , <code>df.select_dtypes()</code> , string methods in accessors	Chapter 15
34	Bonus: 5 new changes in pandas you need to know about	20:54	Deprecations, copy-on-write, nullable integer dtypes	Chapter 9 & 15
35	Bonus: My top 25 pandas tricks	27:37	<code>df.explode()</code> , <code>pd.cut()</code> , <code>pd.qcut()</code> , <code>Series.clip()</code> , <code>style</code>	Chapter 15
36	Bonus: 21 more pandas tricks	24:39	<code>pd.read_clipboard()</code> , filtering with regex, profile inspection	Chapter 15
37	Bonus: Data Science Best Practices with pandas (PyCon 2019)	1:44:16	Method chaining with <code>.pipe()</code> , avoiding <code>apply()</code> traps	Chapter 15
38	Bonus: Your pandas questions answered! (Live Webcast)	1:56:01	Enterprise performance tuning, handling multi-GB ledgers	Chapter 15

Chapter 1: Foundations, Ingestion & Inspection (Videos 1–4)

1. Core Architecture: Series vs DataFrame

A **Series** is a 1D labeled array capable of holding any data type. It is essentially a column of data accompanied by an explicit, immutable row label index. A **DataFrame** is a 2D labeled tabular data structure with columns of potentially different types. You can conceptualize a DataFrame as a collection of Series objects that share a common index.

Hinglish Mental Model

30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

"NumPy ko samjho raw C-style array: fast hai par labeled nahi hai. **pandas** uske upar ek intelligent metadata layer banata hai.

Series ek single column hai with row labels. Jab multiple Series ek hi index share karte hain, toh wo **DataFrame** ban jate hain (jaise SQL table ya Excel sheet)."

Syntax & Reading Tabular Data

```
import pandas as pd
import numpy as np

# Video 1 & 2: Reading delimited files
# Default read_csv assumes comma separation and row 0 as header
df_orders = pd.read_csv("data/chipotle.tsv", sep='\t')

# Reading file with NO header and custom column names
user_cols = ['user_id', 'age', 'gender', 'occupation', 'zip_code']
df_users = pd.read_csv("data/u.user", sep='|', header=None, names=user_cols)

# Ingestion Subsetting (Optimizing memory during read)
df_sample = pd.read_csv("data/u.user", sep='|', names=user_cols, usecols=['user_id', 'age'],
nrows=50)
```

Selecting Columns & Creating Features (Video 3)

```
# Bracket notation (Universal & Recommended)
cities = df_orders['item_name']

# Dot notation (Convenient, BUT fails if column name has spaces or matches DataFrame methods)
cities = df_orders.item_name

# Creating a new combined column (MUST use bracket notation!)
df_orders['order_summary'] = df_orders['quantity'].astype(str) + "x " + df_orders['item_name']
```

Methods vs Attributes Rule (Video 4)

Why does `df.shape` have no parentheses, but `df.head()` does? - **Attributes (No parentheses `()`)**: Describe properties or state that the DataFrame *already has* stored in memory: `df.shape` (returns `(rows, cols)`), `df.dtypes` (data types), `df.columns` (headers), `df.index`. - **Methods (Require parentheses `()`)**: Action verbs that *perform an action, calculation, or transformation*: `df.head()` (slice top rows), `df.describe()` (compute summary statistics), `df.mean()` (aggregate).

Common Traps

- Dot Notation for Assignment:** Writing `df.new_col = [1, 2, 3]` will NOT create a DataFrame column! It merely monkey-patches an attribute onto the Python object. Always use `df['new_col'] = ...`.
- Column Name Collisions:** If a column is named `'count'`, `df.count` accesses the built-in method `DataFrame.count()`, not your data!

? Interview Questions & Follow-ups

- **Q:** "Why does pandas use a separate Series structure instead of standard 1D NumPy arrays?" **A:** While a 1D NumPy array is homogenous and indexed purely by integer offsets, a pandas Series attaches a labeled index, supports heterogeneous data via object pointers, handles `NaN` missing values without crashing, and implements automatic alignment during arithmetic operations.
- **Follow-up:** "What is the memory overhead of a pandas Series vs a NumPy array?" **A:** A NumPy int64 array requires exactly 8 bytes per element in contiguous memory. A pandas Series stores the NumPy array plus an Index object (typically an `Index` or `RangeIndex` with hash map lookups), adding metadata overhead. For string columns, pandas historically stored pointers to Python string objects, incurring up to 5x-8x memory overhead compared to raw C char arrays.

Chapter 2: Schema Manipulation, Renaming & Axis Mechanics (Videos 5, 6, 11)

🧠 Concept: Renaming & Dropping Dimensions

Modifying column headers and discarding unneeded rows or columns is the first step in financial ETL pipelines. Understanding the `axis` parameter is fundamental: operations in pandas can execute vertically down rows (`axis=0` / `'index'`) or horizontally across columns (`axis=1` / `'columns'`).

🧠 Hinglish Mental Model

📌 KEY ENGINEERING INSIGHT

"`axis=0` ka matlab hai: **Rows ke along travel karo (Downwards)**. Jab aap `df.drop(0, axis=0)` karte ho, toh wo row 0 ko drop karta hai. Jab aap `df.mean(axis=0)` karte ho, toh wo saari rows ko collapse karke har column ka 1 average deta hai.

`axis=1` ka matlab hai: **Columns ke along travel karo (Horizontally)**. `df.drop('amount', axis=1)` column ko drop karta hai. `df.mean(axis=1)` har row ke liye horizontally columns ka average nikalta hai."

flowchart TD

```
subgraph Axis0["axis=0 or axis='index' (Downwards across Rows)"]
```

```
  R1["Row 0"] → R2["Row 1"] → R3["Row 2"]
```

```
  R3 → Collapse0["df.mean(axis=0) → 1 scalar per column"]
```

```
end
```

```
subgraph Axis1["axis=1 or axis='columns' (Horizontally across Columns)"]
```

```
  C1["Col A"] → C2["Col B"] → C3["Col C"]
```

```
  C3 → Collapse1["df.mean(axis=1) → 1 scalar per row"]
```

```
end
```

Syntax & Code Examples (Videos 5, 6, 11)

```
import pandas as pd

# Method 1: Selective rename via Dictionary (Safest)
df.rename(columns={'Txn Ref': 'txn_ref', 'Amt (INR)': 'amount'}, inplace=True)

# Method 2: Bulk column rename (Overwriting df.columns)
df.columns = ['txn_id', 'vpa', 'amount', 'timestamp', 'status']

# Method 3: Clean string headers with regex (Production Best Practice)
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# Dropping columns (axis=1)
df.drop(['vpa', 'status'], axis=1, inplace=True)

# Dropping rows by label index (axis=0)
df.drop([0, 1, 5], axis=0, inplace=True)

# The Axis Invariant in Math:
df_financials = pd.DataFrame({
    'q1_profit': [100, 200],
    'q2_profit': [150, 250]
})
# Column-wise average (1 scalar per quarter across all branches):
quarterly_mean = df_financials.mean(axis=0) # or axis='index'

# Row-wise average (1 scalar per branch across both quarters):
branch_mean = df_financials.mean(axis=1) # or axis='columns'
```

Common Traps

- Forgetting `axis=1` when dropping columns: `df.drop('my_col')` defaults to `axis=0` and throws `KeyError: "[ 'my_col' ] not found in axis"` because pandas searched for a row labeled `'my_col'`.

Chapter 3: Sorting, Boolean Filtering & Set Operations (Videos 7–10)

Concept: Boolean Indexing & Predicate Pushdown

Boolean indexing filters a DataFrame by passing a Series of `True` / `False` values matching the length of the DataFrame.

Hinglish Mental Model

KEY ENGINEERING INSIGHT

"Boolean filtering SQL ke `WHERE` clause jaisa hai.

Python ka standard `and` / `or` pure object ki truth value dekhta hai, isliye pandas Series par fail ho jata hai (`ValueError: Truth value of Series is ambiguous`). Pandas mein hamesha bitwise `&` (AND) aur `|` (OR) use karo, aur **har individual condition ko brackets (...) mein wrap karna compulsory hai** kyunki Python mein bitwise operators ki precedence comparison operators (`>`, `=`) se higher hoti hai!"

📄 Syntax & Multi-Criteria Filtering

```
# Video 7: Multi-column sorting
df.sort_values(by=['branch_id', 'amount'], ascending=[True, False], inplace=True)

# Video 8: Single Boolean Filter
high_value = df[df['amount'] ≥ 50000]

# Video 9: Multiple Filter Criteria (& and |)
# WRONG: df[df.amount ≥ 50000 and df.status == 'SUCCESS'] → CRASHES!
# CORRECT:
flagged_txns = df[(df['amount'] ≥ 50000) & (df['status'] == 'FAILED')]

# Set Membership with .isin() (Cleaner and faster than multiple OR conditions)
top_metro_txns = df[df['city'].isin(['Mumbai', 'Bengaluru', 'Delhi', 'Chennai'])]

# Inverse filter (NOT in set via tilde ~ operator)
non_metro = df[~df['city'].isin(['Mumbai', 'Bengaluru', 'Delhi'])]
```

📄 Ingestion Performance & Memory Control (Video 10)

```
# Reading only necessary columns drastically saves memory on multi-GB files
df_light = pd.read_csv("transactions_2026.csv", usecols=['txn_id', 'amount', 'status'],
nrows=100000)

# Iteration Performance Trap:
# BAD (100x slower):
# for idx, row in df.iterrows(): ...
# GOOD (Vectorized):
df['tax'] = df['amount'] * 0.18
```

Chapter 4: Vectorized String Operations (`.str` Accessor) (Video 12)

🧠 Concept: Vectorized String Processing

Python's standard string methods (`.upper()` , `.split()`) only work on individual string scalars. The `.str` accessor allows these methods to be applied element-wise across an entire pandas Series without manual Python `for` loops.

🧠 Hinglish Mental Model

📌 KEY ENGINEERING INSIGHT

"Agar aap `df['vpa'].upper()` likhoge toh error aayega kyunki Series par direct string method nahi hota. `.str` ek bridge hai jo Series ke har element par string method vectorize karke run karta hai: `df['vpa'].str.upper()`."

Banking Examples & Pattern Extraction

```
# Sample Banking Dataset
df = pd.DataFrame({
    'vpa': ['rohan@okhdfcbank', 'priya@icici', 'amit@idfcbank'],
    'narration': ['UPI/409210/Transfer to Sharma', 'NEFT/5512/Salary Oct', 'IMPS/9912/Vendor Bill'],
    'dirty_amount': [' INR 5,000.00 ', ' INR 12,450.50 ', ' INR 750.00 ']
})

# 1. Pattern Matching & Filtering
is_idfc = df['vpa'].str.contains('idfc', case=False)

# 2. String Splitting & Accessing Components
# Extract UPI PSP handle (the domain after '@')
df['psp_handle'] = df['vpa'].str.split('@').str[1] # .str[1] gets second element of each split list!

# 3. String Cleansing & Stripping
df['clean_amount'] = (
    df['dirty_amount']
    .str.replace('INR', '')
    .str.replace(' ', '')
    .str.strip()
    .astype(float)
)
```

Chapter 5: Data Types, Casting & Coercion (Video 13)

Concept: Dtype Casting & Error Coercion

Data imported from text or CSV files often defaults to `object` (string) due to dirty characters or corrupted rows. Converting to native numeric dtypes is mandatory before performing mathematical or statistical operations.

Syntax & Error Coercion

```
# Standard Type Casting
df['account_id'] = df['account_id'].astype(str) # Prevents treating IDs as math numbers
df['is_flagged'] = df['flag_code'].astype(bool)

# The Coercion Superpower: pd.to_numeric
# Suppose an amount column has dirty strings like 'UNKNOWN', 'ERROR', 'REFUND'
raw_amounts = pd.Series(['100.50', '250.00', 'CORRUPT_DATA', '450.75'])

# errors='coerce' turns unparseable values into NaN instead of crashing!
clean_amounts = pd.to_numeric(raw_amounts, errors='coerce')
# Result: [100.50, 250.00, NaN, 450.75]
```

Chapter 6: Series Exploration & Univariate Analysis (Video 15)

Concept: Descriptive Statistics & Frequency Distributions

Univariate inspection allows rapid verification of value distributions, card fraud velocity, and categorical proportions.

Syntax & Examples

```
# Summary statistics for numeric Series (count, mean, std, min, 25%, 50%, 75%, max)
df['amount'].describe()

# Frequency counts of categorical column
channel_counts = df['channel'].value_counts()

# Proportional / Percentage breakdown (normalize=True)
channel_percentages = df['channel'].value_counts(normalize=True) * 100

# Include missing values in count
channel_counts_with_na = df['channel'].value_counts(dropna=False)

# Bivariate Contingency Table (Cross-Tabulation)
# Shows transaction count by Channel across Status in a 2D matrix
pd.crosstab(df['channel'], df['status'], margins=True)
```

Chapter 7: Missing Data Detection, Filtration & Imputation (Video 16)

Concept: NaN Semantics & Imputation Strategies

In pandas, missing values are represented as `np.nan` (IEEE floating-point Not-a-Number) or `pd.NA`. Missing data must either be filtered out or mathematically imputed before machine learning ingestion or ledger calculation.

Hinglish Mental Model

KEY ENGINEERING INSIGHT

"`df.dropna()` data ko delete karta hai, jabki `df.fillna()` missing values ko kisi sensible constant (mean, median, mode ya 'UNKNOWN') se replace karta hai.

Financial transaction ledgers mein kabhi bhi blind `mean` impute nahi karte, kyunki outliers data ko skew kar dete hain; **median** hamesha safer rehta hai."

Syntax & Examples

```
# 1. Detection
missing_per_column = df.isna().sum()
percentage_missing = df.isna().mean() * 100

# 2. Dropping
# Drop row if ANY column is NaN
df_clean = df.dropna(how='any')

# Drop row ONLY if ALL columns are NaN
df_clean = df.dropna(how='all')

# Drop row only if specific critical columns are NaN
df_clean = df.dropna(subset=['account_id', 'amount'], how='any')

# 3. Imputation
# Constant imputation
df['merchant_category'].fillna('UNCLASSIFIED', inplace=True)

# Skew-resistant statistical imputation
median_bal = df['balance'].median()
df['balance'].fillna(median_bal, inplace=True)

# Forward Fill (propagate last valid observation forward - common in stock tick data)
df['stock_price'].ffill(inplace=True)
```

Chapter 8: The Pandas Index & Automatic Arithmetic Alignment (Videos 17–18)

Concept: The Index Invariant & Automatic Alignment

The index in pandas serves three critical functions: 1. **Identification:** Direct row identity independent of position. 2. **Selection:** Fast $O(1)$ hash-based label lookups via `.loc`. 3. **Automatic Alignment:** When performing arithmetic operations between two Series (`s1 + s2`), pandas aligns data on **matching index labels**, NOT on integer position!

Code: Proving Index Alignment

```
# Series 1: Branch revenue in Jan
jan_rev = pd.Series([100, 200, 300], index=['Bengaluru', 'Mumbai', 'Delhi'])

# Series 2: Branch revenue in Feb (Note: different branch ordering and new branch!)
feb_rev = pd.Series([150, 250, 400], index=['Mumbai', 'Bengaluru', 'Hyderabad'])

total_rev = jan_rev + feb_rev
print(total_rev)
# Output:
# Bengaluru    450.0 (100 + 350 aligned!)
# Delhi        NaN (Exists in Jan, missing in Feb → NaN)
# Hyderabad    NaN (Exists in Feb, missing in Jan → NaN)
# Mumbai       350.0 (200 + 150 aligned!)
```

Index Manipulation Syntax

```
# Setting a column as index
df.set_index('account_id', inplace=True)

# Resetting index back to standard default 0, 1, 2... integer range
df.reset_index(inplace=True)
```

Chapter 9: Selection Mastery (`loc` , `iloc`) & The SettingWithCopyWarning (Videos 19, 20, 27)

Concept: Label (`loc`) vs Position (`iloc`) & Views vs Copies

- `df.loc[row_labels, col_labels]` : Purely label-based. **Endpoint is INCLUSIVE!**
- `df.iloc[row_positions, col_positions]` : Purely 0-indexed integer position. **Endpoint is EXCLUSIVE!**

Hinglish Mental Model: The SettingWithCopyWarning Trap

🎤 30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

"Sabse zyada pooche jaane wala interview bug!

Jab aap `df[df.amount > 5000]['status'] = 'FLAGGED'` likhte ho, toh pehle `df[df.amount > 5000]` ek subset return karta hai, aur fir `['status'] = ...` uspar write karta hai. Isko bolte hain **Chained Indexing**.

Pandas guarantee nahi de sakta ki wo intermediate subset underlying memory ka **VIEW** tha ya memory ki nayi **COPY**! Agar wo copy thi, toh original DataFrame update nahi hoga aur data silently corrupt ho jayega.

Golden Fix: Hamesha single-bracket `.loc` use karo: `df.loc[df.amount > 5000, 'status'] = 'FLAGGED'`. Aur agar intentionally slice banana hai, toh explicitly `.copy()` call karo!"

Code: Correct vs Broken Assignment

```
# ❌ DANGEROUS: Causes SettingWithCopyWarning
df[df['amount'] > 100000]['risk_tier'] = 'HIGH'

# ✅ CORRECT: Single-stage label assignment via .loc
df.loc[df['amount'] > 100000, 'risk_tier'] = 'HIGH'

# ✅ CORRECT: Explicit isolated copy
high_net_worth = df[df['balance'] > 5000000].copy()
high_net_worth['relationship_manager'] = 'Senior VP' # Completely safe!
```

Chapter 10: Groupby & Aggregation Mastery (Split-Apply-Combine) (Video 14)

Concept: Split → Apply → Combine

1. **Split:** Divides the dataset into discrete buckets based on unique values in key columns.

2. **Apply:** Computes an aggregation (sum, mean, count) or transformation independently per bucket.
3. **Combine:** Merges results into a single consolidated output DataFrame.

```

flowchart TD
    Raw["Raw Transactions DataFrame"] --> Split["Split by 'account_type'"]
    Split --> S1["S1 SAVINGS Bucket"]
    Split --> S2["S2 CURRENT Bucket"]
    Split --> S3["S3 NRI Bucket"]
    S1 --> Agg1["Compute mean(amount)"]
    S2 --> Agg2["Compute mean(amount)"]
    S3 --> Agg3["Compute mean(amount)"]
    Agg1 --> Combine["Combine into Summary DataFrame"]
    Agg2 --> Combine
    Agg3 --> Combine

```

📖 Advanced Syntax: Named Aggregations

```

# Multi-column groupby with named aggregations (Cleanest Production Syntax)
branch_metrics = df.groupby(['branch_id', 'channel']).agg(
    total_volume=('amount', 'sum'),
    avg_ticket_size=('amount', 'mean'),
    txn_count=('txn_id', 'count'),
    max_single_txn=('amount', 'max')
).reset_index()

```

Chapter 11: Memory Optimization & Categorical Dtypes (Video 21)

🧠 Concept: The `category` Dtype Mechanism

In Python, string columns (`object` dtype) store an array of 8-byte pointers where each pointer references an independent Python string object on the heap. If a column with 10,000,000 transactions contains only 4 unique status values (`'SUCCESS'`, `'FAILED'`, `'PENDING'`, `'REVERSED'`), storing 10M separate string objects consumes hundreds of megabytes.

Converting to `category` creates an integer lookup array (e.g. `uint8` consuming 1 byte per row) pointing to a small 4-element array of strings, reducing memory footprint by **up to 85%–90%**!

📖 Memory Profiling & Optimization Code

```

# Deep memory profiling (must pass memory_usage='deep' to measure string heaps)
print(df.info(memory_usage='deep'))

# Converting low-cardinality string columns to category
df['status'] = df['status'].astype('category')
df['channel'] = df['channel'].astype('category')

# Ordered Categorical (Enables mathematical comparison > and < on categories!)
risk_tiers = pd.CategoricalDtype(categories=['LOW', 'MEDIUM', 'HIGH', 'CRITICAL'],
ordered=True)
df['risk_tier'] = df['risk_tier'].astype(risk_tiers)

# Now you can filter using inequality operators!
urgent_cases = df[df['risk_tier'] >= 'HIGH']

```

Chapter 12: Dates, Times & Time Series Analytics (Video 25)

Concept: The `.dt` Datetime Accessor

Parsing dates into native `datetime64[ns]` enables high-performance component extraction, timestamp math, and time series resampling without manual `strftime` loops.

Syntax & Examples

```
# Parse string to datetime
df['created_at'] = pd.to_datetime(df['created_at'])

# Component extraction via .dt accessor
df['hour'] = df['created_at'].dt.hour
df['day_name'] = df['created_at'].dt.day_name()
df['is_weekend'] = df['created_at'].dt.dayofweek.isin([5, 6])

# Date filtering (String literals auto-convert)
recent_txns = df[df['created_at'] >= '2026-01-01']

# Time Difference / Latency Calculation
# Time elapsed between authorization and settlement
df['settlement_latency_sec'] = (df['settled_at'] - df['authorized_at']).dt.total_seconds()
```

Chapter 13: Deduplication & Feature Engineering (Videos 24, 26)

Concept: Finding & Removing Duplicates & Dummy Variables

- `df.duplicated(subset=[...], keep='first'|'last'|False)`: Identifies duplicate records. Passing `keep=False` flags **all instances of duplicates**, allowing fraud analysts to audit duplicate payment requests.
- `pd.get_dummies(df, drop_first=True)`: Converts categorical variables into numeric binary indicators for statistical models and ML classifiers. `drop_first=True` avoids multicollinearity (the Dummy Variable Trap).

Syntax & Code Examples

```
# 1. Audit Duplicate Payments (keep=False marks all copies)
fraud_duplicates = df[df.duplicated(subset=['sender_acc', 'amount', 'vpa'], keep=False)]

# 2. Safe Deduplication
df.drop_duplicates(subset=['idempotency_key'], keep='first', inplace=True)

# 3. One-Hot Encoding for Machine Learning Credit Scoring
df_encoded = pd.get_dummies(df, columns=['account_type', 'employment_status'], drop_first=True)
```

Chapter 14: Function Application & DataFrame Constructors (Videos 29, 30)

Concept: Function Application Matrix

- `Series.map()` : Best for mapping values using a dictionary or simple 1-to-1 conversion.
- `Series.apply()` : For custom complex transformation on each element of a Series.
- `DataFrame.apply(axis=0|1)` : Applies function across rows or columns.
- `DataFrame.map()` (formerly `applymap`) : Applies function element-wise across every single cell in a DataFrame.

Function Application Examples

```
# 1. Series.map with dictionary (Ultra-fast categorical mapping)
tier_map = {'SAVINGS': 1, 'SALARY': 2, 'WEALTH': 3}
df['tier_code'] = df['account_type'].map(tier_map)

# 2. DataFrame.apply across rows (axis=1)
def calculate_risk_score(row):
    return (row['amount'] / 1000) * (2 if row['is_international'] else 1)

df['risk_score'] = df.apply(calculate_risk_score, axis=1)

# 3. DataFrame constructors from raw Python objects (Video 29)
# Constructor 1: Dictionary of lists
df1 = pd.DataFrame({'id': [1, 2], 'name': ['Aarav', 'Priya']})

# Constructor 2: List of dictionaries (Standard API JSON response)
api_payload = [{'id': 101, 'status': 'SUCCESS'}, {'id': 102, 'status': 'FAILED'}]
df2 = pd.DataFrame(api_payload)
```

Chapter 15: MultiIndex, Merging & Top 46 Productivity Tricks (Videos 31–38)

Concept: MultiIndex Reshaping & Relational Integrity Joins

- **MultiIndex**: Hierarchical row and column labels. `unstack()` converts inner row levels into columns (pivoting), while `stack()` collapses columns into row levels.
- `pd.merge()` : Relational join engine.
- `indicator=True` : Appends `_merge` column (`both`, `left_only`, `right_only`) to immediately audit missing ledger balances.
- `validate='one_to_many'` : Validates relational schema constraints at runtime.

Multindex & Merge Syntax

```
# 1. MultiIndex via Groupby
multi_grouped = df.groupby(['branch_id', 'channel'])['amount'].sum()

# Reshaping: unstack() turns 'channel' index level into columns!
pivoted_table = multi_grouped.unstack()

# 2. Merging with Integrity Validation
merged_ledger = pd.merge(
    df_accounts,
    df_transactions,
    on='account_id',
    how='left',
    indicator=True,
    validate='one_to_many'
)

# Identify accounts with ZERO transactions:
inactive_accounts = merged_ledger[merged_ledger['_merge'] == 'left_only']
```

Top 10 Power Tricks (Videos 33–38)

```
# 1. Readable SQL-style querying
df_filtered = df.query('amount > 50000 and status == "SUCCESS"')

# 2. Exploding list-valued columns into individual rows
df_exploded = df.explode('transaction_tags')

# 3. Numerical Binning (Fixed Bins)
df['bracket'] = pd.cut(df['amount'], bins=[0, 1000, 25000, 100000], labels=['Micro', 'Retail', 'HNI'])

# 4. Quantile Binning (Equal Count Bins)
df['quartile'] = pd.qcut(df['amount'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

# 5. Numerical Outlier Clipping
df['clipped_amount'] = df['amount'].clip(lower=100, upper=100000)

# 6. Read tabular data straight from operating system clipboard!
# df_clipboard = pd.read_clipboard()
```

Quick Revision: The 10 Golden pandas Rules for Interviews

1. **loc vs iloc**: **loc** is label-based (inclusive boundary); **iloc** is integer position (exclusive boundary).
2. **axis Parameter**: **axis=0** acts downwards across rows; **axis=1** acts horizontally across columns.
3. **Compound Filtering**: Use **&** and **|** with mandatory parentheses: `df[(df.a > 0) & (df.b == 1)]`.
4. **Prevent SettingWithCopyWarning**: Never chain assignments (`df[cond]['col'] = x`). Always use `df.loc[cond, 'col'] = x`.
5. **Memory Optimization**: Convert low-cardinality string columns to **category** dtype for 80%+ RAM reduction.
6. **Error Coercion**: Use `pd.to_numeric(df.col, errors='coerce')` to gracefully turn corrupt strings into **NaN**.

7. **Datetime Power:** Convert via `pd.to_datetime()` and extract components via `.dt` accessor (`.dt.hour`, `.dt.day_name()`).
8. **Deduplication Audits:** Use `df.duplicated(subset=[...], keep=False)` to view all conflicting transactions.
9. **Index Alignment:** Series arithmetic aligns on **index labels**, not array order. Mismatched labels produce `NaN`.
10. **Avoid `iterrows()`:** Vectorize calculations or use NumPy/`.groupby()`. Row-by-row iteration in Python is 50x–100x slower.

CHAPTER 04 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Attributes (No parentheses `()`): Describe properties or state that the DataFrame **already has** stored in memory:
- Methods (Require parentheses `()`): Action verbs that **perform an action, calculation, or transformation**:
- Q: **"Why does pandas use a separate Series structure instead of standard 1D NumPy arrays?"**

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

Matplotlib: Data Visualization & Financial Plotting

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



Matplotlib: Data Visualization & Financial Plotting

1. What is Matplotlib & Why Does It Exist?

Simple Language Mein:



KEY ENGINEERING INSIGHT

"Data ko tables ya numbers mein dekh kar human brain patterns quickly catch nahi kar pata. Agar fraud transactions spike ho rahe hain ya monthly card spending badh rahi hai, toh **Visual Charts** sabse tezi se insights dete hain.

Matplotlib Python ki foundational 2D plotting library hai. pandas ke internal visualization methods (`df.plot()`) aur Seaborn sabhi under the hood Matplotlib par hi chalte hain."

2. The Anatomy of a Figure: Figure vs Axes vs Axis

```
flowchart TD
    subgraph FigureCanvas["Figure (The Complete Canvas / Window)"]
        subgraph Ax1["Axes 1 (Plot Sub-window 1)"]
            T1["Title & Legend"]
            Plot1["Line Plot / Bar Chart"]
            XAxis1["X-Axis (Ticks, Labels)"]
            YAxis1["Y-Axis (Ticks, Labels)"]
        end
        subgraph Ax2["Axes 2 (Plot Sub-window 2)"]
            Plot2["Histogram / Scatter"]
        end
    end
end
```

 30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

[!IMPORTANT]

 Flashcard: Figure vs Axes

- **Figure:** Complete image ya canvas (page size, background).
- **Axes:** Actual graph ya plotting area (jisme x-axis, y-axis, lines, bars, title hote hain). Ek Figure mein multiple Axes (subplots) ho sakte hain!
- **Axis:** Specific number line (X-axis horizontal ya Y-axis vertical jo ticks aur limits define karti hai).

3. Two Interfaces: Pyplot (Stateful) vs Object-Oriented API

Approach A: Pyplot Interface (`plt.plot()`)

Matlab-style global state machine. Simple scripts ke liye theek hai, lekin complex subplots mein confusing hota hai:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 20, 25])
plt.title("Simple Line")
# plt.show()
```

Approach B: Object-Oriented API (`fig, ax = plt.subplots()`) — RECOMMENDED!

Explicitly Figure aur Axes objects banate hain. Production dashboards aur multi-panel charts ke liye industry standard hai:

```
fig, ax = plt.subplots(figsize=(8, 4)) # Width: 8 inches, Height: 4 inches
ax.plot([1, 2, 3], [10, 20, 25], color="darkred", marker="o", label="Daily Spend")
ax.set_title("IDFC Customer Spend Trend", fontsize=14, fontweight="bold")
ax.set_xlabel("Days")
ax.set_ylabel("Amount (₹)")
ax.legend()
ax.grid(True, linestyle="--", alpha=0.6)
# fig.savefig("spend_trend.png")
```

4. The 5 Essential Chart Types in Banking

```
import matplotlib.pyplot as plt
import numpy as np

# Sample Data
categories = ["Food", "Shopping", "Travel", "Utilities", "Healthcare"]
spend = [18500, 42000, 12500, 8400, 15000]
months = ["Jan", "Feb", "Mar", "Apr", "May"]
monthly_inflow = [50000, 52000, 48000, 65000, 70000]
tx_amounts = np.random.exponential(scale=2500, size=500)

# Create 2x2 multi-panel figure
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 1. Line Chart: Time-Series Trend
axes[0, 0].plot(months, monthly_inflow, marker="s", color="#9b1c1c", linewidth=2)
axes[0, 0].set_title("Monthly Salary Inflow Trend")
axes[0, 0].grid(True)

# 2. Bar Chart: Categorical Comparison
axes[0, 1].bar(categories, spend, color="#2b6cb0")
axes[0, 1].set_title("Spend by Merchant Category")
axes[0, 1].tick_params(axis="x", rotation=30)

# 3. Histogram: Distribution of Transaction Amounts
axes[1, 0].hist(tx_amounts, bins=25, color="#38a169", edgecolor="black")
axes[1, 0].set_title("Transaction Value Distribution")
axes[1, 0].set_xlabel("Amount (₹)")

# 4. Scatter Plot: Transaction Amount vs Fraud Risk Score
risk_scores = np.random.uniform(0, 100, 500)
axes[1, 1].scatter(tx_amounts, risk_scores, alpha=0.5, color="#d69e2e")
axes[1, 1].set_title("Amount vs Anomaly Risk Score")
axes[1, 1].set_xlabel("Amount")
axes[1, 1].set_ylabel("Risk (0-100)")

plt.tight_layout() # Prevents label overlapping!
# plt.savefig("dashboard.png")
```

5. Matplotlib Interview Question Bank (20 Questions)

1. Figure aur Axes mein kya difference hai?

Answer: Figure pura outer canvas/window hota hai; Axes actual plot area hota hai jisme data points, ticks, aur labels render hote hain.

2. Why is the Object-Oriented API (`fig, ax = plt.subplots()`) preferred over `plt.plot()` ?

Answer: OO API explicit references deta hai, multiple subplots ko cleanly manage karta hai, aur global state bugs avoid karta hai.

3. What is the purpose of `plt.tight_layout()` ?

Answer: Automatically adjusts subplots padding to prevent overlapping titles and axis labels.

4. How do you save a chart as an image without showing a GUI popup window?

Answer: `fig.savefig('chart.png', dpi=300, bbox_inches='tight')`.

5. How do you run Matplotlib on a headless Linux server without a display/monitor?

Answer: Use non-interactive backend before importing pyplot: `python import matplotlib`
`matplotlib.use('Agg') import matplotlib.pyplot as plt`

6. When should you use a Histogram vs a Bar Chart?

Answer: Bar chart categorical discrete data compare karta hai (e.g., spending by category);
 Histogram continuous numerical data ki frequency distribution dikhata hai (e.g., frequency of amounts between ₹100 and ₹1,000).

7. What is the difference between a Scatter plot and a Line plot?

8. How do you add annotations or callout text to highlight an outlier on a plot?

Answer: `ax.annotate('Suspicious Spike!', xy=(x, y), xytext=(x+1, y+500),`
`arrowprops=dict(facecolor='red'))` .

9. How do you plot twin axes (e.g., primary Y-axis for Transaction Volume and secondary Y-axis for Fraud Percentage)?

Answer: `ax2 = ax1.twinx()` .

10. What parameter controls marker transparency in scatter plots to handle overplotting?

Answer: `alpha` parameter (e.g., `alpha=0.3`).

11. How do you customize X-axis date formatting in time-series plots?

Answer: `import matplotlib.dates as mdates;`
`ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))` .

12. How do you rotate tick labels so they don't overlap?

Answer: `ax.tick_params(axis='x', rotation=45)` or `plt.xticks(rotation=45)` .

13. How do you plot a horizontal bar chart?

Answer: `ax.barh(categories, values)` .

14. How do you change the color palette or style sheet in Matplotlib?

Answer: `plt.style.use('seaborn-v0_8-whitegrid')` or `plt.style.use('ggplot')` .

15. What is a Box Plot (`ax.boxplot()`) and what 5 summary metrics does it display?

Answer: Minimum, 25th percentile (Q1), Median, 75th percentile (Q3), Maximum, and Outliers beyond $1.5 * IQR$.

16. How do you add a horizontal benchmark line across a chart?

Answer: `ax.axhline(y=50000, color='red', linestyle='--')` .

17. How do you control the resolution of saved image files?

Answer: The `dpi` parameter in `savefig()` (e.g., `dpi=300` for publication/print quality).

18. How do you close figures to prevent memory leaks in long-running services?

Answer: `plt.close(fig)` or `plt.close('all')` . If you don't close figures in a loop, memory keeps accumulating in the GUI backend.

19. How do you create a pie chart with percentage labels displayed?

Answer: `ax.pie(values, labels=labels, autopct='%1.1f%%')` .

20. How does pandas integrate with Matplotlib?

Answer: `df.plot(kind='bar', ax=ax)` directly attaches pandas plot rendering onto existing Matplotlib Axes objects.

 CHAPTER 05 SUMMARY & REVISION CHECKLIST **WHAT TO REMEMBER**

- Core fundamentals and architectural mechanics of Matplotlib: Data Visualization & Financial Plotting
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

 CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 06

SQL pandas Rosetta Stone: Translating SQL Knowledge to pandas

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



SQL ↔ pandas Rosetta Stone: Translating SQL Knowledge to pandas

1. Why This Matters

Aapka SQL background already solid hai. Nayi syntax zero se yaad karne ke bajaye, agar aap **SQL queries ko direct pandas operations se map** kar loge, toh aap 1 ghante ke andar pandas mein expert level analysis likhne lagoge!

2. The Core Conceptual Mapping

```
flowchart LR
    subgraph RelationalSQL["Relational Database (SQL)"]
        Table["Table"]
        Row["Row / Record"]
        Column["Column / Field"]
        PrimaryKey["Primary Key"]
    end

    subgraph PythonPandas["pandas Ecosystem"]
        DF["DataFrame"]
        SeriesRow["Index Entry"]
        SeriesCol["Series"]
        Index["DataFrame Index"]
    end

    Table <--> DF
    Row <--> SeriesRow
    Column <--> SeriesCol
    PrimaryKey <--> Index
```

3. Clause-by-Clause Translation Table

1. **SELECT** (Column Projection)

SQL

```
SELECT transaction_id, amount
FROM transactions;

SELECT DISTINCT merchant_category
FROM transactions;
```

pandas

```
# Select columns
df[["transaction_id", "amount"]]

# DISTINCT
df["merchant_category"].unique() # or df[["merchant_category"]].drop_duplicates()
```

2. **WHERE** (Filtering & Predicates)

SQL

```
SELECT * FROM transactions
WHERE amount > 10000 AND status = 'SUCCESS';

SELECT * FROM transactions
WHERE merchant_category IN ('FOOD', 'TRAVEL');

SELECT * FROM transactions
WHERE merchant_category IS NULL;
```

pandas

```
# Multiple conditions (& for AND, | for OR, ~ for NOT)
df[(df["amount"] > 10000) & (df["status"] == "SUCCESS")]

# IN condition (isin)
df[df["merchant_category"].isin(["FOOD", "TRAVEL"])]

# IS NULL / IS NOT NULL
df[df["merchant_category"].isna()]
df[df["merchant_category"].notna()]
```

3. **ORDER BY** & **LIMIT** (Sorting & Pagination)

SQL

```
SELECT * FROM transactions
ORDER BY amount DESC, timestamp ASC
LIMIT 10 OFFSET 20;
```

pandas

```
# sort_values()
df.sort_values(
    by=["amount", "timestamp"],
    ascending=[False, True]
).iloc[20:30] # Limit 10 Offset 20
```

4. GROUP BY & HAVING (Aggregations)

SQL

```
SELECT
    merchant_category,
    COUNT(*) AS total_txns,
    SUM(amount) AS total_spend,
    AVG(amount) AS avg_spend
FROM transactions
GROUP BY merchant_category
HAVING SUM(amount) > 100000;
```

pandas

```
# groupby() + agg()
category_summary = df.groupby("merchant_category").agg(
    total_txns=("amount", "count"),
    total_spend=("amount", "sum"),
    avg_spend=("amount", "mean")
).reset_index()

# HAVING clause filter:
category_summary[category_summary["total_spend"] > 100000]
```

5. JOIN (Relational Merging)

SQL

```
-- INNER JOIN
SELECT t.*, c.first_name, c.phone
FROM transactions t
INNER JOIN customers c ON t.customer_id = c.customer_id;

-- LEFT OUTER JOIN
SELECT t.*, c.first_name
FROM transactions t
LEFT JOIN customers c ON t.customer_id = c.customer_id;
```

pandas

```
# INNER JOIN
pd.merge(transactions_df, customers_df, on="customer_id", how="inner")

# LEFT JOIN
pd.merge(transactions_df, customers_df, on="customer_id", how="left")

# RIGHT JOIN: how='right'
# FULL OUTER JOIN: how='outer'
```

6. UNION ALL & UNION (Vertical Concatenation)

SQL

```
SELECT * FROM transactions_jan
UNION ALL
SELECT * FROM transactions_feb;
```

pandas

```
# UNION ALL
pd.concat([jan_df, feb_df], axis=0, ignore_index=True)

# UNION (with deduplication)
pd.concat([jan_df, feb_df], axis=0, ignore_index=True).drop_duplicates()
```

7. Window Functions (OVER (PARTITION BY ... ORDER BY ...))

SQL Running Balance

```
SELECT
    transaction_id,
    account_id,
    amount,
    SUM(amount) OVER (
        PARTITION BY account_id
        ORDER BY timestamp
    ) AS running_balance
FROM transactions;
```

pandas Equivalent

```
# Groupby + cumsum()
df["running_balance"] = df.sort_values("timestamp").groupby("account_id")["amount"].cumsum()
```

SQL Ranking (`ROW_NUMBER()` / `DENSE_RANK()`)

```
SELECT
  *,
  DENSE_RANK() OVER (
    PARTITION BY customer_id
    ORDER BY amount DESC
  ) AS rank_in_customer
FROM transactions;
```

pandas Equivalent

```
# rank(method='dense')
df["rank_in_customer"] = df.groupby("customer_id")["amount"].rank(
    method="dense",
    ascending=False
)
```

SQL `LAG()` / `LEAD()` (Previous / Next row)

```
SELECT
  account_id,
  amount,
  LAG(amount, 1) OVER (PARTITION BY account_id ORDER BY timestamp) AS prev_amount
FROM transactions;
```

pandas Equivalent

```
# shift(1) for LAG, shift(-1) for LEAD
df["prev_amount"] = df.sort_values("timestamp").groupby("account_id")["amount"].shift(1)
```

8. `CASE WHEN` (Conditional Column Assignment)

SQL

```
SELECT
  transaction_id,
  amount,
  CASE
    WHEN amount ≥ 50000 THEN 'HIGH'
    WHEN amount ≥ 10000 THEN 'MEDIUM'
    ELSE 'LOW'
  END AS risk_category
FROM transactions;
```

pandas (Vectorized with `np.select`)

```
import numpy as np

conditions = [
    df["amount"] >= 50000,
    df["amount"] >= 10000
]
choices = ["HIGH", "MEDIUM"]

df["risk_category"] = np.select(conditions, choices, default="LOW")
```

4. Quick Summary Checklist for Interviews

- `SELECT col` ⇒ `df['col']` or `df[['a', 'b']]`
- `WHERE` ⇒ `df[(df['col'] > x) & (df['col2'] == y)]`
- `GROUP BY` ⇒ `df.groupby('col').agg(...)`
- `HAVING` ⇒ filter the aggregated DataFrame output
- `JOIN` ⇒ `pd.merge(df1, df2, on='key', how='inner|left|outer')`
- `UNION ALL` ⇒ `pd.concat([df1, df2], axis=0)`
- `SUM() OVER(PARTITION BY ...)` ⇒ `df.groupby('col')['val'].cumsum()`
- `CASE WHEN` ⇒ `np.select(conditions, choices, default=...)`

CHAPTER 06 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- ``SELECT col`` ⇒ ``df['col']`` or ``df[['a', 'b']]``
- ``WHERE`` ⇒ ``df[(df['col'] > x) & (df['col2'] == y)]``
- ``GROUP BY`` ⇒ ``df.groupby('col').agg(...)``

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 07

The Complete Python Data Analysis Workflow: NumPy pandas Matplotlib

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



The Complete Python Data Analysis Workflow: NumPy → pandas → Matplotlib

1. The Real-World Engineering Pipeline

In production banking backends and analytics microservices, raw transactional data often flows from database queries, CSV dumps, or message queues into an analytical pipeline:

```
flowchart TD
    Raw["1. Raw Data Source (transactions.csv / SQL Dump)"] --> Load["2. Load into pandas DataFrame (pd.read_csv / pd.read_sql)"]
    Load --> Clean["3. Data Cleaning (Impute NaNs, drop duplicates, parse timestamps)"]
    Clean --> Vectorize["4. NumPy Vectorization (Z-score anomaly detection, thresholding)"]
    Vectorize --> Agg["5. pandas Aggregation (Split-Apply-Combine groupby & pivots)"]
    Agg --> Viz["6. Matplotlib Visualization (Multi-panel dashboard figure)"]
    Viz --> Output["7. Export Clean Data & Executive Dashboard PNG"]
```

2. Step-by-Step Code Walkthrough (Banking Transaction Analytics)

Step 1: Load Data into pandas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("01-PYTHON/transactions.csv")
print(f"Loaded {len(df)} transactions. Shape: {df.shape}")
```

Step 2: Clean and Impute Missing Values

```
# Check missing counts
print(df.isna().sum())

# Impute numerical missing amounts with the median to avoid skewing:
median_amount = df["amount"].median()
df["amount"] = df["amount"].fillna(median_amount)

# Impute categorical missing values:
df["merchant_category"] = df["merchant_category"].fillna("UNKNOWN")

# Convert timestamp strings to true pandas DateTime:
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["date"] = df["timestamp"].dt.date
```

Step 3: Beginner Analytics (Key KPI Summary)

```
total_volume = df["amount"].sum()
avg_ticket = df["amount"].mean()
max_txn = df["amount"].max()
status_counts = df["status"].value_counts()

print(f"Total Settled: ₹{total_volume:,.2f}")
print(f"Avg Ticket: ₹{avg_ticket:,.2f}")
print(f"Status Distribution:\n{status_counts}")
```

Step 4: Intermediate Analytics (GroupBy Spend by Category)

```
category_summary = df.groupby("merchant_category").agg(
    volume=("amount", "sum"),
    txn_count=("transaction_id", "count"),
    avg_ticket=("amount", "mean")
).sort_values("volume", ascending=False)

print(category_summary)
```

Step 5: Advanced Anomaly Detection Using NumPy Vectorization

```
# Statistical Outlier Detection: Z-Score > 3 (Mean + 3 * StdDev)
mean_val = np.mean(df["amount"])
std_val = np.std(df["amount"])
anomaly_cutoff = mean_val + (3 * std_val)

# Vectorized boolean mask
anomalies = df[df["amount"] > anomaly_cutoff]
print(f"Anomaly Cutoff: > ₹{anomaly_cutoff:,.2f}")
print(f"Flagged Suspicious Transactions: {len(anomalies)}")
```

Step 6: Visual Dashboard Generation with Matplotlib OO API

```
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
fig.suptitle("IDFC FIRST Bank – Transaction Health Dashboard", fontsize=16, fontweight="bold",
color="#9b1c1c")

# 1. Daily Volume Trend (Line Chart)
daily = df.groupby("date")["amount"].sum()
axes[0, 0].plot(daily.index, daily.values, color="#9b1c1c", linewidth=2)
axes[0, 0].set_title("Daily Transaction Volume (₹)")
axes[0, 0].grid(True, linestyle="--", alpha=0.5)

# 2. Category Spend (Bar Chart)
axes[0, 1].bar(category_summary.index, category_summary["volume"] / 1000, color="#2b6cb0")
axes[0, 1].set_title("Volume by Category (₹ '000)")
axes[0, 1].tick_params(axis="x", rotation=30)

# 3. Success vs Failure (Pie Chart)
axes[1, 0].pie(status_counts.values, labels=status_counts.index, autopct="%1.1f%", colors=
["#38a169", "#e53e3e", "#dd6b20"])
axes[1, 0].set_title("Transaction Status Ratio")

# 4. Distribution with Anomaly Cutoff (Histogram)
axes[1, 1].hist(df["amount"], bins=30, color="#4a5568", edgecolor="white")
axes[1, 1].axvline(anomaly_cutoff, color="red", linestyle="--", label=f"Anomaly (>₹
{int(anomaly_cutoff):,})")
axes[1, 1].set_title("Transaction Amount Distribution")
axes[1, 1].legend()

plt.tight_layout()
fig.savefig("01-PYTHON/banking_analytics_dashboard.png", dpi=300)
plt.close(fig)
```

3. How to Run the Live Pipeline

You can run this full pipeline anytime directly in your terminal:

```
python3 01-PYTHON/banking-analytics-project.py
```

It loads [01-PYTHON/transactions.csv](#), computes all analytics, and updates [01-PYTHON/banking_analytics_dashboard.png](#).



CHAPTER 07 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of The Complete Python Data Analysis Workflow: NumPy pandas Matplotlib
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 08

Most Asked & Tricky Python Interview Questions

PRIORITY

 High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Most Asked & Tricky Python Interview Questions

The Senior Engineer Playbook: CPython Internals, GIL, Metaclasses, Closures & Dunder Puzzles

 KEY ENGINEERING INSIGHT**Target Role:** Senior / Core Developer (3+ Years Experience) | Strategic Projects, Bengaluru**Prerequisites:** Python 3.10+, OOP, Memory Layout, Concurrency Models.**Key Focus:** Bytecode disassembly, memory allocation pitfalls, GIL internals, and FinTech data bugs.

Quick Navigation

1. [Small Integer Caching, String Interning & Memory Identity \(`is` vs `==` \)](#)
2. [The Mutable Default Argument Trap & Bytecode Analysis](#)
3. [Late-Binding Closures in Lambdas & Loops](#)
4. [The Tuple Mutation Mystery \(`t\[0\] += \[4, 5\]` \)](#)
5. [CPython GIL, Threading vs Multiprocessing vs Asyncio](#)
6. [Method Resolution Order \(MRO\) & The Diamond Problem](#)
7. [`\_\_new\_\_` vs `\_\_init\_\_` & Writing Thread-Safe Singletons](#)
8. [`\_\_slots\_\_` Memory Optimization in High-Frequency Ledgers](#)
9. [Context Managers: Suppressing Exceptions in `\_\_exit\_\_`](#)
10. [Chained Comparisons & Tricky Boolean Logic](#)

1. Small Integer Caching, String Interning & Memory Identity (`is` vs `==`)

Why Interviewers Ask This

Tests whether you understand how CPython manages memory on the heap and how memory pooling works for performance.

🧩 Tricky Code Puzzle

What does this print, and why?

```
a = 256
b = 256
print(a is b)

x = 257
y = 257
print(x is y)
```

✅ Output & Explanation

True
False (In interactive REPL / separate code blocks)

🧠 CPython Memory Internals & Hinglish Mental Model

📌 KEY ENGINEERING INSIGHT

"Python me integers immutable objects hote hain. Bar-bar naye integer objects create karne ke CPU overhead se bachne ke liye CPython startup par ek static array pre-allocate karta hai: **Small Integer Cache: -5 se 256 tak.**

Jab aap `a = 256` aur `b = 256` likhte ho, dono exactly same pre-allocated `PyLongObject` memory address ko point karte hain (`a is b → True`). Lekin `257` is cache range ke bahar hai! CPython heap par do alag memory blocks allocate karta hai. Value same hai (`x = y → True`), par identity alag hai (`x is y → False`)."

⚠️ Note on Code Block Compiler Optimization

If you run `x = 257; y = 257` in the same Python module or `.py` script, CPython's code block compiler / AST optimizer may intern duplicate constants within the same code object, causing `x is y` to evaluate to `True`. But relying on `is` for value equality is a severe production bug.

🏠 Production Rule

- Always use `=` for data value comparison (amounts, customer IDs, status codes).
- Use `is` **strictly** for singleton identity comparisons (`val is None`, `flag is True`).

2. The Mutable Default Argument Trap & Bytecode Analysis

💡 Why Interviewers Ask This

One of the top 3 most frequently asked Python traps. Evaluates whether you know *when* function definitions are evaluated in Python.

🧩 Tricky Code Puzzle

What does this code output?

```
def record_transaction(txn_id, ledger=[]):
    ledger.append(txn_id)
    return ledger

print(record_transaction("TXN-001"))
print(record_transaction("TXN-002"))
print(record_transaction("TXN-003", []))
print(record_transaction("TXN-004"))
```

✗ Intuitive (Wrong) Answer

```
['TXN-001']
['TXN-002']
['TXN-003']
['TXN-004']
```

✓ Actual Output

```
['TXN-001']
['TXN-001', 'TXN-002']
['TXN-003']
['TXN-001', 'TXN-002', 'TXN-004']
```

🧠 Root Cause: When are default arguments evaluated?

Default parameter expressions are evaluated **once**, at the time the `def` statement is executed (when the module is loaded), NOT on each function call! The default list object is stored directly in the function object's `__defaults__` attribute:

```
print(record_transaction.__defaults__) # ([ 'TXN-001', 'TXN-002', 'TXN-004' ],)
```

Any call that does not supply a second argument shares and mutates this **exact same list in memory**.

🔧 Production Fix: The `None` Sentinel Pattern

```
def record_transaction(txn_id, ledger=None):
    if ledger is None:
        ledger = [] # Fresh list created in local stack frame on every call
    ledger.append(txn_id)
    return ledger
```

3. Late-Binding Closures in Lambdas & Loops

💡 Why Interviewers Ask This

Tests your understanding of Python's lexical scoping and variable lookup mechanism during closure execution.

🧩 Tricky Code Puzzle

What does this output?

```
multipliers = [lambda x: x * i for i in range(4)]
results = [m(2) for m in multipliers]
print(results)
```

❌ Intuitive (Wrong) Answer

`[0, 2, 4, 6]` (assuming each lambda captures `i = 0, 1, 2, 3`).

✅ Actual Output

`[6, 6, 6, 6]`

🧠 The Mechanics: Late Binding

Python's closures are **late-binding**: variables used in closures are looked up when the inner function is **called**, not when it is defined! When `m(2)` is called, the list comprehension has already completed, and the variable `i` in the enclosing scope equals `3`. Therefore, every lambda computes `2 * 3 = 6`.

🔧 Production Fix: Default Argument Binding

Default arguments are evaluated at **function definition time**. We can bind the current loop value into a local default parameter:

```
multipliers = [lambda x, i=i: x * i for i in range(4)]
print([m(2) for m in multipliers]) # [0, 2, 4, 6]!
```

4. The Tuple Mutation Mystery (`t[0] += [4, 5]`)

💡 Why Interviewers Ask This

A legendary Python brain-teaser that exposes the boundary between mutable contained objects and immutable outer containers.

🧩 Tricky Code Puzzle

What happens when you run this code?

```
t = ([1, 2], 3)
try:
    t[0] += [4, 5]
except TypeError as e:
    print(f"Caught exception: {e}")

print("Final tuple:", t)
```

❌ Common Guesses

1. "It raises `TypeError` and the tuple remains `([1, 2], 3)` unchanged."

2. "It succeeds without error and becomes `([1, 2, 4, 5], 3)`."

✓ Actual Shocking Answer

Both happen! An exception is raised, **AND** the list inside the tuple is mutated!

Caught **exception**: 'tuple' object does **not** support item assignment
Final **tuple**: `([1, 2, 4, 5], 3)`

🧠 Bytecode Disassembly: Why does this happen?

Let's inspect the compiled CPython bytecode using `dis.dis("t[0] += [4, 5]")`:

```
1. BINARY_SUBSCR    → Fetches t[0] (the list [1, 2])
2. LOAD_CONST       → Loads [4, 5]
3. INPLACE_ADD      → Calls list.__iadd__([4, 5]). Mutates the list in-place!
4. STORE_SUBSCR     → Tries to assign the result back: t[0] = mutated_list
```

Step 3 executes `list.extend()` successfully, modifying the list in-place on the heap. Then Step 4 attempts to assign the reference back to `t[0]`. Because `tuple` is immutable, Step 4 raises `TypeError: 'tuple' object does not support item assignment`!

🚨 Production Lesson

Never place mutable containers (lists, dicts) inside immutable collections (tuples, frozensets) if you plan to modify them using augmented assignment operators (`+=`, `*=`).

5. CPython GIL, Threading vs Multiprocessing vs Asyncio

💡 The Core Question

"What is the GIL, why can't Python threads achieve true parallelism on multi-core CPUs, and when should you choose Multiprocessing vs Threading vs Asyncio?"

🧠 Architecture Breakdown

The **Global Interpreter Lock (GIL)** is a mutual exclusion lock used by CPython to prevent multiple native OS threads from executing Python bytecode simultaneously.

Why does the GIL exist? CPython's memory management relies heavily on **Reference Counting**. Without a global lock, concurrent threads would race to increment/decrement `ob_refcnt` on shared `PyObject` structs, leading to race conditions, memory leaks, or premature deallocations.

 The Concurrency Decision Matrix

Model	Module	Best Suited For	Uses Multiple Cores?	Overhead
Multithreading	<code>threading</code>	I/O-bound tasks (Network calls, DB queries, Disk reads)	No (GIL limits bytecode execution to 1 core at a time)	Low (threads share process memory)
Multiprocessing	<code>multiprocessing</code>	CPU-bound computation (Cryptographic hashing, ML models, large data transforms)	Yes (Each process has its own memory space & independent GIL)	High (Inter-Process Communication & memory serialization via <code>pickle</code>)
Asynchronous I/O	<code>asyncio</code>	High-concurrency cooperative I/O (10,000+ open WebSockets, API gateways)	No (Single-threaded cooperative event loop)	Minimal (No thread context-switching overhead)

 FinTech Interview Curveball: Does NumPy use the GIL?

No! NumPy, Pandas C-extensions, and SciPy release the GIL during heavy vectorized SIMD matrix computations. While doing heavy array math, NumPy runs across multiple native C threads in parallel across all CPU cores.

6. Method Resolution Order (MRO) & The Diamond Problem

 Why Interviewers Ask This

Tests whether you understand Python's **C3 Linearization Algorithm** for multiple inheritance and how `super()` delegates calls dynamically.

✖ Tricky Diamond Inheritance Puzzle

```
class A:
    def ping(self):
        print("A")

class B(A):
    def ping(self):
        print("B")
        super().ping()

class C(A):
    def ping(self):
        print("C")
        super().ping()

class D(B, C):
    def ping(self):
        print("D")
        super().ping()

d = D()
d.ping()
```

✖ Intuitive (Wrong) Answer

"D -> B -> A -> C -> A" (Calling A twice).

✓ Actual Output

```
D
B
C
A
```

🧠 Hinglish Mental Model: How `super()` Works

🔑 KEY ENGINEERING INSIGHT

"`super()` ka matlab 'mera direct parent' nahi hota! `super()` ka matlab hota hai: 'MRO list me mere agle bhai/parent ko call karo!'"

Python `Class.mro()` check karke deterministic order banata hai: `D.mro() → [D, B, C, A, object]`

Jab `B` ke andar `super().ping()` chalta hai, wo `A` ko call nahi karta! Wo MRO me `B` ke baad aane wali class `C` ko call karta hai. Is tarah class `A` sirf **ek baar** run hoti hai (Diamond problem solved)!"

```
print([cls.__name__ for cls in D.mro()])
# ['D', 'B', 'C', 'A', 'object']
```

7. `__new__` vs `__init__` & Writing Thread-Safe Singletons

💡 The Difference

- `__new__(cls, ...)`: The actual **Constructor**. A static method responsible for allocating the object in memory and returning the new instance.
- `__init__(self, ...)`: The **Initializer**. Receives the newly created instance and initializes its state/attributes.

💻 Production Thread-Safe Singleton (Database Connection Pool)

```
import threading

class DatabaseConnectionPool:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            with cls._lock: # Double-Checked Locking Pattern
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, dsn="postgresql://core_bank:5432"):
        # Prevent re-initialization on subsequent __new__ calls
        if not hasattr(self, '_initialized'):
            self.dsn = dsn
            self._initialized = True
```

8. `__slots__` Memory Optimization in High-Frequency Ledgers

💡 Why Senior Interviewers Ask This

When processing millions of transaction objects per second, standard Python classes allocate a dynamic dictionary (`__dict__`) for every instance, costing ~150 to 200 bytes of heap memory per object.

🧠 How `__slots__` Solves This

By declaring `__slots__ = ('txn_id', 'amount', 'timestamp')`:

1. Python eliminates the per-instance `__dict__` and `__weakref__`.
2. Memory is allocated as a fixed C-style struct array of pointers.
3. Memory drops by **40% to 60%**, and attribute access speed increases by ~20%.

```
class StandardTxn:
    def __init__(self, id, amt):
        self.id = id
        self.amt = amt

class OptimizedTxn:
    __slots__ = ('id', 'amt') # Pre-allocated fixed slots
    def __init__(self, id, amt):
        self.id = id
        self.amt = amt

# In a 5-million transaction ledger, OptimizedTxn saves ~600MB of RAM!
```

9. Context Managers: Suppressing Exceptions in `__exit__`

💡 The Tricky Mechanics

In a custom Context Manager (`__enter__` and `__exit__`), if an exception occurs inside the `with` block, Python passes the exception details (`exc_type` , `exc_val` , `exc_tb`) to `__exit__` .

- If `__exit__` returns `True` , Python **suppresses** the exception and continues normal execution!
- If `__exit__` returns `False` (or `None`), Python **re-raises** the exception.

🖥️ Safe Ledger Transaction Context Manager

```
class LedgerTransactionScope:
    def __init__(self, session):
        self.session = session

    def __enter__(self):
        self.session.begin()
        return self.session

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            self.session.rollback()
            # If exc_type is a non-fatal business retry, return True to suppress:
            if isinstance(exc_type, TransientNetworkError):
                print(f"Handled transient error: {exc_val}. Suppressing.")
                return True # Exception suppressed!
            return False # Fatal error re-raised
        self.session.commit()
        return True
```

10. Chained Comparisons & Tricky Boolean Logic

🧩 Tricky Brain-Teaser 1: Chained Comparison

What does this print?

```
print(False == False in [False])
```

✗ Intuitive (Wrong) Answer

"True == in [False] -> TypeError" or "True in [False] -> False"

✓ Actual Output

True!

🧠 Why?

In Python, comparisons chain mathematically like `a < b < c` expands to `(a < b) and (b < c)`. Therefore:

```
False == False in [False]
# Expands to:
(False == False) and (False in [False])
# Evaluates to:
True and True → True!
```

✂ Tricky Brain-Teaser 2: try ... finally Return Override

What does this function return?

```
def check_balance():
    try:
        return 100
    finally:
        return 200

print(check_balance())
```

✓ Output: 200

The `finally` block is guaranteed to execute before the function leaves the stack frame. If the `finally` block executes an explicit `return` or `raise`, it **silently discards** any return value or unhandled exception originating from the `try` block!

🧠 CHAPTER 08 SUMMARY & REVISION CHECKLIST

💡 WHAT TO REMEMBER

- Always use ``==`` for data value comparison (amounts, customer IDs, status codes).
- Use ``is`` strictly for singleton identity comparisons (``val is None``, ``flag is True``).
- ``__new__(cls, ...)``: The actual Constructor. A static method responsible for allocating the object in memory and returni

✓ CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 09

Last-Minute Python Data Ecosystem Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

⚡ Last-Minute Python Data Ecosystem Cheatsheet

🐍 Python in 5 Minutes

1. Types & Mutability

- **Immutable (Cannot change in-place):** `int`, `float`, `str`, `tuple`, `frozenset`, `bytes`.
- **Mutable (In-place changes):** `list`, `dict`, `set`, `bytearray`.
- **List vs Tuple:** List mutable hai (`[1, 2]`); Tuple immutable hai (`(1, 2)`) - faster, hashable, can be dict keys).
- **Dict vs Set:** Dict key-value map hai; Set unique keys only hai (both $O(1)$ avg lookups).

2. Copying & References

- `b = a` $\Rightarrow$ Same pointer (mutating `b` mutates `a`).
- `shallow = a.copy()` $\Rightarrow$ New outer container, shared nested objects.
- `deep = copy.deepcopy(a)` $\Rightarrow$ Completely independent recursive clone.

3. Comprehensions & Packing

- `[x*2 for x in arr if x > 0]` (List) | `{k: v for k, v in pairs}` (Dict)
- `def f(*args, **kwargs)` : `*args` captures tuple, `**kwargs` captures dictionary.

4. Generators, Decorators & Context Managers

- **Generator** (`yield`): Lazy evaluation, produces 1 item at a time in $O(1)$ memory.
- **Decorator** (`@func`): Wraps another function to add logging/auth without altering code.
- **Context Manager** (`with`): Guarantees `__exit__()` cleanup on DB connections/files.

1 2 3 4 NumPy in 3 Minutes

1. ndarray Anatomy

- **Contiguous memory:** C-style contiguous buffer of homogeneous bytes (unlike scattered Python lists).

- **shape** : Dimension tuple (e.g., `(2, 3)` = 2 rows, 3 cols).
- **axis=0** : Down rows (column-wise). **axis=1** : Across cols (row-wise).
- **dtype** : Data type (`int64`, `float32`).

2. Slicing & Reshape

- **Slices are Views**: Modifying `arr[1:3]` modifies the original array! Use `.copy()` if you need independence.
- **reshape(3, -1)** : `-1` auto-calculates remaining dimension.
- **flatten()** : Always returns a copy. **ravel()** : Returns a view when possible.

3. Broadcasting & Vectorization

- **Broadcasting Rules**: Dimensions match or one of them is 1 (stretches along size 1).
- **Vectorization**: Replaces slow Python loops with hardware SIMD C-level CPU instructions (40x–80x faster).

pandas in 5 Minutes

1. Core Structures

- **Series**: 1D labeled array. **DataFrame**: 2D labeled table of Series.

2. Selection: `loc` vs `iloc`

- **df.loc['row', 'col']** : **Label-based** (inclusive of end boundary).
- **df.iloc[0:5, 0:2]** : **Integer-position based** (0-indexed, exclusive of end boundary).

3. Filtering & Cleaning

- Multiple filters: `df[(df['amt'] > 1000) & (df['status'] == 'SUCCESS')]` (use `&` and `|` with parentheses).
- Missing Data: `df.isna().sum()` | `df.dropna()` | `df['amt'].fillna(df['amt'].median())`.
- Duplicates: `df.drop_duplicates(subset=['txn_id'])`.

4. GroupBy & Merging

- **Split-Apply-Combine**: `python df.groupby('category')['amt'].agg(['sum', 'mean', 'count']).reset_index()`
- **SQL Merges**: `python pd.merge(df1, df2, on='cust_id', how='left')` # SQL: LEFT JOIN
`pd.concat([df1, df2], axis=0)` # SQL: UNION ALL
- **Avoid `iterrows()`** : Vectorize with `np.select` or list comprehensions instead of loops.

Matplotlib in 2 Minutes

1. Anatomy: Figure vs Axes

- **Figure:** The overall canvas/window.
- **Axes:** The actual individual plot/graph (x-axis, y-axis, data points, title).

2. Object–Oriented Interface (Recommended)

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, color="#9b1c1c", label="Trend")
ax.set_title("IDFC Transaction Volume")
ax.set_xlabel("Date")
ax.set_ylabel("Volume (₹)")
ax.grid(True, linestyle="--")
ax.legend()
plt.tight_layout()
fig.savefig("chart.png", dpi=300)
plt.close(fig)
```

3. Chart Types

- **Line** (`ax.plot`): Time-series trends.
- **Bar** (`ax.bar`): Categorical comparisons.
- **Histogram** (`ax.hist`): Numerical distribution.
- **Scatter** (`ax.scatter`): Relationship / correlations between two variables.

CHAPTER 09 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Immutable (Cannot change in-place): ``int``, ``float``, ``str``, ``tuple``, ``frozenset``, ``bytes``.
- Mutable (In-place changes): ``list``, ``dict``, ``set``, ``bytearray``.
- List vs Tuple: List mutable hai (``[1, 2]``); Tuple immutable hai (``(1, 2)`` - faster, hashable, can be dict keys).

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 10

pandas Last-Minute Revision: High-Yield Cram Sheets

PRIORITY

 High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

pandas Last-Minute Revision: High-Yield Cram Sheets

Designed for quick review before technical interviews at IDFC FIRST Bank.

5-Minute Emergency Review

- **Selection Core:**
- `df.loc[rows, cols]` ⇒ **Label-based** (both start and stop labels are **INCLUSIVE**).
- `df.iloc[rows, cols]` ⇒ **0-indexed integer position** (start is inclusive, stop is **EXCLUSIVE**).
- **Axis Invariant:**
- `axis=0` / `'index'` ⇒ Moves **downwards across rows** (collapses rows into 1 value per column).
- `axis=1` / `'columns'` ⇒ Moves **horizontally across columns** (collapses columns into 1 value per row).
- **Multiple Boolean Filter:**
- `df[(df['amount'] > 1000) & (df['status'] == 'SUCCESS')]` (Brackets `(...)` are mandatory!).
- **Prevent `SettingWithCopyWarning`:**
- Never use chained indexing: `df[df['a'] > 0]['b'] = 10` ❌
- Always use `.loc`: `df.loc[df['a'] > 0, 'b'] = 10` ✅
- **Groupby Split-Apply-Combine:**
- `df.groupby('branch')['amount'].agg(['count', 'sum', 'mean'])`.

15-Minute Review

- **Data Ingestion Subsetting:**
- `pd.read_csv('ledger.csv', usecols=['id', 'amount'], nrows=10000)` saves memory upfront.
- **Vectorized String Accessor (`.str`):**
- `df['vpa'].str.split('@').str[1]` extracts PSP handle without Python loops.
- `df['narration'].str.contains('SALARY', case=False)`.

- **Date Handling (`.dt`):**
 - `df['ts'] = pd.to_datetime(df['ts'])`
 - `df['ts'].dt.hour`, `df['ts'].dt.day_name()`, `df['ts'].dt.dayofweek`.
 - **Type Coercion:**
 - `pd.to_numeric(df['dirty_col'], errors='coerce')` turns unparseable values into `NaN`.
 - **Deduplication:**
 - `df.duplicated(keep=False)` marks ALL copies as True (crucial for fraud audits).
 - `df.drop_duplicates(subset=['idempotency_key'], keep='first')`.
 - **Relational Merge:**
 - `pd.merge(df1, df2, on='acc_id', how='left', indicator=True)` audits join drops via `_merge` column.
-



30–Minute Deep Revision

- **Memory Optimization (`category` Dtype):**
 - String columns store 8–byte pointers to individual Python heap objects.
 - `df['channel'] = df['channel'].astype('category')` replaces strings with 1–byte integer codes and a category dictionary, reducing memory usage by up to 85%.
 - Check true memory usage with: `df.info(memory_usage='deep')`.
 - **Index Alignment in Arithmetic:**
 - When computing `s1 + s2`, pandas aligns on **index labels**, NOT on integer position!
 - If a label exists in `s1` but not `s2`, the result is `NaN`.
 - **Function Application Matrix:**
 - `Series.map()`: Element-wise using a dictionary or single-argument mapper.
 - `Series.apply()`: Element-wise using complex lambda or custom function.
 - `DataFrame.apply(axis=0|1)`: Runs function across each column or row.
 - `DataFrame.map()` (formerly `applymap`): Runs function element-wise on every cell in a 2D table.
 - **Multindex Operations:**
 - `unstack()`: Pivots innermost index level into column headers.
 - `stack()`: Collapses column headers into innermost index level.
 - `df.xs('MUMBAI', level='city')`: Selects cross-section from Multindex.
-

1-Hour Comprehensive Interview Drill

10 Rapid-Fire Code Output Puzzles

- Puzzle 1: `loc` slicing with integer index** `python s = pd.Series([10, 20, 30, 40], index=[3, 2, 1, 0]) print(s.loc[2:0])` # Answer: Returns 20, 30, 40 (label-based slice starting from label 2 to label 0 inclusive!)
- Puzzle 2: Bitwise filtering without brackets** `python # What error does df[df.amount > 1000 & df.status == 'A'] throw? # Answer: TypeError / ValueError because & has higher operator precedence than > and ==.`
- Puzzle 3: Series addition with mismatched indexes** `python a = pd.Series([1, 2], index=['x', 'y']) b = pd.Series([3, 4], index=['y', 'z']) print((a + b)['x'])` # Answer: NaN (Index 'x' does not exist in Series b).
- Puzzle 4: Drop axis default** `python # What does df.drop('col_name') do? # Answer: Throws KeyError because axis defaults to 0 (rows), not columns!`
- Puzzle 5: Creating columns with dot notation** `python df.new_col = [1, 2, 3]` # Did this add a column to df? # Answer: NO. It sets a monkey-patched Python attribute on the DataFrame object. \n

CHAPTER 10 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Selection Core:
- ``df.loc[rows, cols]`` $\Rightarrow$ Label-based (both start and stop labels are INCLUSIVE).
- ``df.iloc[rows, cols]`` $\Rightarrow$ 0-indexed integer position (start is inclusive, stop is EXCLUSIVE).

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases